AUREON PRODUCTION-GRADE RECONCILIATION RULE ENGINE SPECIFICATION

Status: Implementation-Ready Design Document
Version: 1.0
Audience: Engineering team (strong backend, non-finance expertise)
Context: Multi-tenant SaaS reconciliation platform for PMS/AIF/hedge funds/custodians/wealth managers

EXECUTIVE SUMMARY

This document specifies a **three-tier hybrid reconciliation engine** for Aureon:

- **Tier-1 (Deterministic):** SQL-safe, exact-match rules with fixed tolerances (~70% of breaks)
- **Tier-2 (Tolerance-based):** Amount, date, FX variance thresholds (~20% of breaks)
- **Tier-3 (AI-Reasoning):** Fuzzy semantic matching, narratives, partial fills, edge cases (~10% of breaks)
- **Self-Healing:** Log manual fixes → analyze patterns → update rule memory → improve future coverage

The engine will:

1. Ingest normalized trade, cash, position, NAV, corporate action, and fee data from multiple sources
2. Run reconciliation rules grouped into **7 domains** with **120+ rule specifications**
3. Escalate unresolved breaks to ops teams with rich context
4. Learn from manual overrides to reduce future false positives

Key outputs:

- Rule catalog (detailed specs for 120+ rules across 7 domains)
- Exception workflows (lifecycle, severity, routing)
- Tier-1 / Tier-2 / Tier-3 partitioning strategy
- Python rule engine API (25+ core functions)
- Self-healing architecture
- Phased 12-week implementation roadmap

SECTION 0 — SCOPE & ASSUMPTIONS

0.1 Scope Definition

Target Institutions:

- **Primary:** PMS (Portfolio Management Services - India), AIFs (Alternative Investment Funds), hedge funds
- **Secondary:** Mutual funds (simpler), custodians, wealth managers, family offices
- **All:** Managing client portfolios (not standalone trading desks)

Asset Classes (MVP Phase 1):

- Listed equities (NSE/BSE in India; NYSE/NASDAQ in US; LSE in UK)
- ETFs (equity-tracking)
- Mutual funds (units)
- Bonds (basic plain-vanilla, not complex structures)
- Cash (INR, USD, EUR, GBP)
- Simple derivatives: equity futures, index futures, currency forwards (limited)
- NOT in scope (Phase 1): complex derivatives (swaps, swaptions, exotic options), commodities, crypto

Geographies & Regulatory Context (Tier-1 focus):

Region	Regulator	Settlement	Key Rules	Notes
India	SEBI	T+1 (equities), T+2 (bonds)	NSE/BSE norms; DP (depository) via NSDL/CDSL	All PMS/AIF rules apply
US	SEC/FINRA	T+1 (equities as of 2024)	DTC, NSCC norms; T+2 legacy still common	Covered in Phase 2
UK/EU	FCA/ESMA	T+2 historically (may change)	Euroclear/Clearstream; EMIR reporting	Covered in Phase 2
APAC (ex-India)	MAS/SFC/SGX	T+2 (Singapore, HK)	Local clearing; HKEX, SGX norms	Covered in Phase 2

Phase 1 deep-dive: India PMS + US hedge funds (T+1/T+2 context)

0.2 Explicit Assumptions

A. Data Availability

Guaranteed to exist (by time rules run):

Table	Key Fields	Assumptions
`broker_trades`	trade_id, trade_date, settlement_date, symbol, isin, quantity, price, gross_amount, net_amount, side, currency, broker, status, source_file	One row per trade leg; already normalized by ingestion; fees parsed into net_amount
`bank_txns`	txn_id, value_date, booking_date, description, amount, running_balance, currency, status, counterparty, source_system	One row per cash movement; debit/credit already encoded in amount sign
`holdings`	position_id, as_of_date, symbol, isin, quantity, cost_base, market_price, total_value, currency, source (CUSTODIAN|INTERNAL)	Snapshot per day per symbol; one row per position
`nav_logs`	nav_date, fund_isin, nav_per_unit, total_aum, total_units, currency, source (CUSTODIAN|INTERNAL|COMPUTE)	One row per fund per date; maybe multiple calculations (custodian vs internal)
`corp_actions`	id, ex_date, record_date, symbol, isin, type (SPLIT|BONUS|DIVIDEND|RIGHTS|MERGER), ratio_numerator, ratio_denominator, cash_amount, currency	Pre-loaded by subscription feeds or manual entry
`broker_fees`	fee_id, trade_id, booking_date, fee_type (BROKERAGE|STT|GST|EXCHANGE|CUSTODY|OTHER), amount, currency, status	May be separate from trade, or embedded in gross/net

Partially available (with Tier-0 clean-up):

- Broker-supplied narratives / descriptions (used by AI, Tier-3)
- Cross-currency rates (end-of-day custodian rates preferred; fallback to market feeds)
- Corporate action details (may require cross-reference with data provider)

NOT assumed:

- Real-time positions (end-of-day batch only)
- Settlement live feeds (batch files, ~1–2 per day)
- Live cash ledger; we work with periodic (daily) bank statements

B. Frequency & Timing

Batch cycles (Phase 1):

- **Daily EOD batch:** Run all reconciliation for T (once per day, ~8 PM IST for India)
- **Intraday micro-batches:** When a new file is uploaded (for demo/monitoring)
- **Trigger-based:** When a CA notification arrives (corporate action adjustments)

Settlement cycle:

- **India:** T+1 for equities; T+2 for bonds, FX (starting post-Jan 2024)
- **US:** T+1 for equities (post-May 2024); previously T+2
- Phase 1 rules assume T+1/T+2 context; adaptable per region

Reconciliation windows:

- Trade ↔ Cash: from T+0 to T+3 (to catch delayed settlement)
- Positions ↔ Custodian: from T+1 to T+5 (post-settlement lag + admin delay)
- NAV checks: T+0 evening (after market close, before fund close)

C. Corporate Action Coverage (Phase 1)

Type	Coverage	Notes
Splits / Reverse Splits	Full	Applied to quantity, price, cost base; auto-adjust positions & match new trades
Bonus Issues	Full	Add to quantity; zero cost; auto-reconcile
Cash Dividends	Basic	Detect via bank entry; match to ex-date position; basic PnL impact
Rights Issues	Limited	Detect; flag for manual assignment logic (Phase 2+)
Mergers / Demergers	Not in Phase 1	Complex multi-leg; defer to Phase 2
Stock Dividends	Basic	Same logic as bonus; quantity change + possible CG implications

CA rules assume:

- Ex-date, record-date, payment-date pre-populated (via data vendor or manual entry)
- Ratio provided (e.g., 5:1 split) or denominator/numerator
- Handled before trade ↔ position reconciliation on the ex-date

SECTION 1 — RECONCILIATION DOMAINS & FLOWS

1.1 Core Reconciliation Domains

The Aureon reconciliation universe is partitioned into 7 core domains:

Domain A: Trade ↔ Cash

What: Match broker trade confirmations against bank/custodian cash movements.
Typical sources:

- Broker trade files (XLSX, PDF contract notes, API confirmations)
- Bank ledger (statements, SWIFT 940s, internal bank files)
- Custodian settlement reports

Typical data flow:

```
Broker Trade (T) → Trade Table
   ↓ (after T+0/T+1)
Settlement expected → Custodian report
   ↓
Bank cash debit/credit (T+1/T+2) → Bank Txns Table
   ↓
Reconciliation Rule: Match trade.net_amount ↔ bank_txn.amount
```

What "good" looks like:

- 100% of settled trades matched to cash entries ±tolerance
- Timing: cash entry appears ±1–2 days of settlement_date (depending on region, asset class)
- Amount: exact or within tolerance (rupees paise, cents, tolerance for FX rounding)

Typical breaks:

- Trade settles but no cash entry (missing bank file, pending)
- Cash entry but no matching trade (incoming credit from dividend, prior-day settlement)
- Amount mismatch (price correction, partial fill not flagged, FX rounding)
- Date mismatch (T+2 vs T+3 settlement, banking holiday)
- Cancellation/reversal not reflected in both systems

Domain B: Positions ↔ Custodian (Holdings)

What: Reconcile internal position book (quantity, value) against custodian/DP holdings statement.
Typical sources:

- Internal position table (built from trade engine + corporate actions)
- Custodian DP statement (NSE depository report, DTCC holdings, Euroclear, etc.)
- Broker/dealer statement (for non-custody holdings, OTC derivatives)

Typical data flow:

```
Internal Trades (cumulative) + Corp Actions → Internal Position Table
   ↓
T+1/T+2 (after settlement) → Holdings Table (internal)
   ↓
Custodian publishes EOD holdings (DP statement) → Holdings Table (custodian source)
   ↓
Reconciliation Rule: internal.quantity ↔ custodian.quantity (per isin)
                     internal.value ↔ custodian.value (using match price or custodian pri
```

What "good" looks like:

- Quantity matches to the unit
- Cost basis matches (or internal cost base is tracked separately from custodian cost)

- Market value matches (using agreed pricing source)
- Haircuts/pledges accounted for (pledged qty != free qty)

Typical breaks:

- Quantity mismatch (missing trade, reversed trade not processed)
- Missing positions (newly bought security not yet in custodian system)
- Extra positions (custodian holding not in internal book; e.g., dividend reinvestment)
- Valuation mismatch (internal using stale price, custodian using different price source)
- Corporate action not reflected (split quantity not updated internally)

Domain C: NAV & AUM Valuation Checks

What: Reconcile fund-level NAV (per unit) and AUM (total) between internal calculations and custodian/admin valuations.
Typical sources:

- Internal NAV compute (Aureon calc engine or other back-office)
- Custodian NAV report (official NAV for PMS/AIF/MF)
- Admin / registrar NAV (for MF)

Typical data flow:

```
Holdings (EOD) → Internal NAV Compute
   ├─ value_date market prices (from market data feeds)
   ├─ add: accrued interest, dividends receivable
   ├─ subtract: management fees, brokerage, taxes
   → Internal NAV per unit

Custodian NAV Feed → Custodian NAV per unit

Reconciliation Rule: |internal.nav - custodian.nav| / custodian.nav < threshold%
                     (e.g., 0.5% or 1% depending on fund policy)
```

What "good" looks like:

- NAV variance < 0.5% for large funds (< 1% acceptable for many PMS)
- AUM variance explainable by fee timing, accrual differences, or market price source differences
- All material assets priced (no "NAV not available for ticker X")
- Illiquid / stale positions flagged separately (not blended into NAV calc)

Typical breaks:

- NAV mismatch > tolerance (usually due to missing holdings, wrong pricing, or fee timing)
- Corporate action adjustment not done (ex-date NAV spike/drop)

- Cash position mismatch at custodian (unsettled trades or pending fees)
- Accrual mismatch (interest, dividend timing)

Domain D: Fees & Charges

What: Reconcile fees charged (management fees, brokerage, STT, GST, custody fees, etc.) between broker/custodian bills and internal accounting.
Typical sources:

- Broker fee notes (brokerage, exchange fees, STT/GST for India)
- Custodian invoices (custody fees, fund admin fees)
- Internal expense tracker

Typical data flow:

```
Broker trades → Embedded fees (STT, GST, brokerage) in net_amount
             OR separate fee entry in broker_fees table
   ↓
Bank cash statement → Fee lines (often lumped as one big debit)
   ↓
Custodian monthly invoice → Lump-sum custody/admin fee
   ↓
Reconciliation Rule: sum of internal fee records ↔ bank ledger fee lines
                     + verify fee type, date, amount per broker/custodian contract
```

What "good" looks like:

- All broker fees (per trade) reconciled to bank ledger
- Monthly custody/admin fees reconciled to custodian invoices
- Fee rates aligned with fund's approved rate sheet
- Tax impact (GST, if applicable) accounted for

Typical breaks:

- Brokerage not charged (discount negotiated?)
- Custody fee variance (fund size changed, annual true-up pending)
- STT not applied or applied incorrectly (India: varies by asset)
- GST mismatch on fees (slab change, invoice timing)
- Duplicate fee entries

Domain E: Corporate Actions

What: Reconcile the impact of corporate actions (splits, bonus, dividends, rights, mergers) on holdings, cost basis, and PnL.
Typical sources:

- Corporate action notification (from exchange, data provider, custodian)
- Internal CA register
- Impact on holdings post ex-date
- Cash received (dividend) or quantity adjustment

Typical data flow:

```
Exchange / Data Provider → CA Notification
   (ex_date, record_date, type, ratio, cash/new_qty)
   ↓
Apply CA Logic:
  - Adjust internal holdings quantity (split 5:1 → qty *= 5)
  - Adjust cost base (split → divide original cost by ratio)
  - Add expected cash (dividend) to expected cash table
  ↓
Holdings (ex-date onward) → Reflect new qty
Custodian Holdings (ex-date onward) → Reflect new qty
Bank Txns (record-date onward) → Dividend cash in or out
   ↓
Reconciliation Rule: Compare holdings pre/post CA; check dividend cash match
```

What "good" looks like:

- Holdings quantity adjusts correctly for splits/bonus
- Cost basis tracked correctly (critical for tax / PnL)
- Dividend cash matched to bank ledger
- Ex-date holdings = pre-ex qty × ratio; no gap
- Position not duplicated or lost

Typical breaks:

- Split quantity not updated (internal or custodian mismatch)
- Dividend not received (yet) / delayed
- Ratio mismatch (sourced from multiple providers, inconsistency)
- Cost basis not adjusted post-split
- Rights offer: allocation not matched to trade

Domain F: FX & Multi-Currency

What: Reconcile trades, cash, holdings, and valuations when dealing with multiple currencies (e.g., INR fund with USD holdings, FX forwards).
Typical sources:

- Trade currency (trade booked in USD, INR, EUR, etc.)
- Settlement currency (may differ from trade currency; FX forward?)
- Base currency (fund reporting currency; usually INR for India PMS, USD for US funds)
- FX rates (custodian rates, market rates, deal rates)

Typical data flow:

```
Trade: BUY 100 Apple @ USD 150 on 1-Jan (trade_ccy=USD) → net_amount_usd = 15,000
        Fund base_ccy = INR
        Spot rate 1-Jan: USD 1 = INR 83.50 → net_amount_inr = 1,252,500

Bank settlement: Debit INR 1,252,500 (T+1)
                 per FX forward deal at USD 1 = INR 83.45 (locked rate)

Reconciliation Rule:
  - Check trade booked in correct currency
  - Check bank debit matches: 100 × 150 × spot_or_deal_rate
  - Track FX variance: (83.45 vs 83.50 spot) × quantity = "cost" of FX deal
```

What "good" looks like:

- FX deals (if hedged) matched to forward contracts
- Spot trades use market rates; variance < tolerance (e.g., 0.05%)
- Conversion to base currency for NAV is consistent (custodian rate used, or internal rate agreed)
- Gains/losses on FX movements tracked separately from P&L

Typical breaks:

- FX rate mismatch (trade executed at spot, but bank used different rate)
- Currency not specified on trade (ambiguous settlement)
- Base currency conversion missing (NAV calc)
- Unmatched FX forwards (trade not settled with hedge)
- Missing FX deal ticket (trade vs forward mismatch)

Domain G: Suspense & Exception Management

What: Manage "orphan" cash, securities, or position discrepancies that don't fit standard reconciliation patterns.
Typical sources:

- Cash in bank but no matching trade (dividend interim payment? Fee error? Reversal pending?)
- Security position in custodian but no trade record (gift? CMA? transfer?)
- Cash mismatch > tolerance (rounding, timing, or genuine error?)
- Trades in limbo (cancelled but not reversed? Partial fill pending?)

Typical data flow:

```
Unmatched entities → Suspense register
   ├─ unmatched_cash (no corresponding trade)
   ├─ unmatched_security (no corresponding trade)
   ├─ reconciliation_break (trade ↔ cash mismatch, or qty mismatch)
   ↓
Exception lifecycle:
  - Auto-investigation (timing? corporate action? known vendor quirk?)
  - Flag for ops review (1-day, 3-day, 7-day aging)
  - Manual resolution (e.g., ops creates offsetting entry, or rules override)
  - Auto-close once resolved
```

What "good" looks like:

- Suspense $ trending to zero (most items resolved same-day or T+3)
- No orphaned cash aging >7 days without explanation
- Security suspense aged by reason (known CA, pending trade, etc.)
- Clearly audit trail (what was done, who approved)

Typical breaks:

- Cash received but source unknown (interim dividend? Fee reversal?)
- Partial fill: buy 1000 shares but only 900 settled (100 pending)
- Reversal: trade marked cancelled, but cash already moved
- Pledge/unpledge: security moved to/from margin account (not a loss, but reconciliation needs logic)

1.2 Domain Snapshot Summary

Domain	Key Reconciliation	Typical Breaks	Severity if Broken
A: Trade ↔ Cash	Broker trade amount vs bank debit; timing	Missing cash; amount mismatch; date lag	CRITICAL (PnL + AUM impact)

Domain	Key Reconciliation	Typical Breaks	Severity if Broken
B: Positions ↔ Custodian	Internal qty/val vs DP holdings	Qty mismatch; missing security; valuation diff	CRITICAL (AUM + client reporting)
C: NAV & Valuation	Internal NAV vs custodian NAV	NAV variance >threshold; missing pricing	CRITICAL (client NAV, regulatory)
D: Fees & Charges	Broker fees vs bank ledger; custodian invoices	Missing fee; wrong fee type; overcharge	HIGH (fund P&L, compliance)
E: Corporate Actions	CA impact on qty, cost, dividends	Qty not adjusted; dividend missed; cost basis wrong	HIGH (PnL, tax compliance)
F: FX & Multi-Ccy	Trade ccy vs settlement ccy vs base ccy	FX rate mismatch; missing hedge; conversion error	HIGH (NAV, PnL)
G: Suspense	Orphaned cash / securities	Unmatched cash; unmatched security; breaks >tolerance	MEDIUM → HIGH (depends on $ amount)

SECTION 2 — DETAILED RULE CATALOG (HEART OF DESIGN)

This section specifies **110+ production-grade rules** across the 7 domains, organized as:

- **Tier-1 Deterministic Rules** (SQL-safe, exact logic)
- **Tier-2 Tolerance-based Rules** (amount, date, FX thresholds)
- **Tier-3 Data Quality Rules** (field validation, impossible states)
- **Edge-Case Rules** (partial fills, reversals, corporate action days)

Each rule follows this structure:

```
- id: DOMAIN_SHORTCODE_###
  domain: <domain name>
  name: <human-readable title>
  type: DETERMINISTIC | TOLERANCE | DATA_QUALITY | EDGE_CASE
  tier: 1 | 2 | 3
  description: <2–3 sentence clear explanation>
  inputs:
    - <field_name>: <meaning>
  logic_pseudocode: |
    <SQL-like or Python pseudocode>
  suggested_tolerances:
    amount_abs: <absolute tolerance in rupees / currency unit>
    amount_pct: <percentage tolerance>
    date_offset_days: <max days difference>
    qty_tolerance: <if quantity-based>
  region_applicability: GLOBAL | INDIA | US | EU | APAC
  rule_classification:
    tier: TIER_1_DETERMINISTIC | TIER_2_TOLERANCE | TIER_3_DATA_QUALITY | EDGE_CASE
    should_live_in: RULE_ENGINE | AI_LAYER | HYBRID
    notes: <why this classification>
  severity:
    level: CRITICAL | HIGH | MEDIUM | LOW
    impact: <regulatory, PnL, client reporting, ops load, etc.>
```

```
    auto_escalate: true | false (if true, requires ops review)
  example_breaks:
    - <realistic scenario 1>
    - <realistic scenario 2>
  example_fix:
    - <how ops resolves today>
  tags: [CORE, OPTIONAL, PHASE_1, PHASE_2, ...]
```

DOMAIN A: TRADE ↔ CASH (20 Rules)

A1: Exact Trade-to-Cash Match

```
- id: TRADE_CASH_001
  domain: Trade ↔ Cash
  name: Exact Trade-to-Cash Amount Match (Settled Trades)
  type: DETERMINISTIC
  tier: 1
  description: |
    For every broker trade marked SETTLED, find a corresponding bank cash entry with
    matching net_amount (or gross_amount if fees embedded) within the expected settlement
    window. This is the primary identity: a settled trade MUST have a cash movement.
  inputs:
    - trade.trade_id: unique identifier
    - trade.settlement_date: T, T+1, T+2 per asset/region
    - trade.net_amount: after all fees, taxes
    - trade.status: SETTLED | UNSETTLED | CANCELLED
    - trade.currency: trade settlement currency
    - bank_txn.value_date: cash posting date
    - bank_txn.amount: cash debit (if sell) or credit (if buy)
    - bank_txn.currency: cash currency
  logic_pseudocode: |
    for each trade where status == SETTLED:
      expected_cash_amount = trade.net_amount
      expected_window_start = trade.settlement_date
      expected_window_end = trade.settlement_date + 2  # T+2 safety margin

      matching_cash = (
        bank_txns
        where amount == expected_cash_amount
          and currency == trade.currency
          and value_date in [expected_window_start, expected_window_end]
          and not yet_matched_to_another_trade
      ).first()

      if matching_cash is None:
        status = UNMATCHED_BREAK
      else:
        create link: trade.id <-> cash.id
        status = MATCHED
  suggested_tolerances:
    amount_abs: 0  # Exact match expected (1 rupee / 1 cent = BREAK)
    amount_pct: 0
    date_offset_days: 2
```

```
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_1_DETERMINISTIC
    should_live_in: RULE_ENGINE
    notes: |
      Bread-and-butter reconciliation rule. Must be in deterministic engine because:
      - Exact match is an accounting identity
      - Relies only on normalized numeric fields
      - High-volume, high-pass-through rate (95%+)
  severity:
    level: CRITICAL
    impact: Direct PnL impact; every unsettled/unmatched trade overstates cash/AUM
    auto_escalate: true
  example_breaks:
    - "Trade settled for INR 1,00,000 but no bank cash entry within T+2"
    - "Trade settled for INR 1,00,000 but bank entry shows INR 99,999 (1 rupee mismatch)'
    - "Trade settled for INR 1,00,000 on T+1, but cash entry only on T+4 (delayed settlem
  example_fix:
    - "Ops checks broker confirmation: if settled, escalate to custodian (may be pending
    - "Ops finds INR 99,999 was net of a previously unknown fee; creates fee adjustment e
    - "Ops checks settlement calendar: T+4 is expected if banking holiday on T+2; rule to
  tags: [CORE, PHASE_1, MUST_PASS]
```

A2: Partial Fill Handling

```
- id: TRADE_CASH_002
  domain: Trade ↔ Cash
  name: Partial Fill Detection and Matching
  type: EDGE_CASE
  tier: 2
  description: |
    When a single broker order is filled in multiple legs (e.g., 1000 share order filled
    match each trade leg independently to its corresponding cash. Total cash must equal s
    Flag if a leg is cancelled mid-way (qty cancelled != 0) and is not reversed in bank.
  inputs:
    - trade.trade_id: leg identifier
    - trade.parent_order_id: broker order number (groups all legs)
    - trade.quantity: qty in this leg
    - trade.quantity_cancelled: if any cancellation
    - trade.net_amount: per leg
    - trade.execution_time: timestamp per leg
    - bank_txn.description: may include "partial" or leg reference
  logic_pseudocode: |
    # Group trades by parent_order_id
    for each parent_order_id:
      total_qty_ordered = trades[parent_order_id].first().original_quantity
      total_qty_filled = sum(trades[parent_order_id].quantity where status != CANCELLED)
      total_qty_cancelled = sum(trades[parent_order_id].quantity_cancelled)

      if total_qty_filled + total_qty_cancelled != total_qty_ordered:
        flag INCOMPLETE_ORDER (qty discrepancy)

      total_expected_cash = sum(trades[parent_order_id].net_amount)
```

```
      matched_cash_legs = []
      for each trade_leg in trades[parent_order_id]:
        cash_leg = find_cash_entry(
          amount == trade_leg.net_amount,
          description contains broker_reference or leg_id,
          value_date ~ settlement_date
        )
        if cash_leg:
          matched_cash_legs.append(cash_leg)

      total_matched_cash = sum(matched_cash_legs.amount)

      if abs(total_matched_cash - total_expected_cash) > tolerance:
        flag PARTIAL_FILL_MISMATCH
  suggested_tolerances:
    amount_abs: 1  # 1 rupee / 1 cent per leg
    amount_pct: 0.01  # 0.01%
    date_offset_days: 3
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_2_TOLERANCE
    should_live_in: RULE_ENGINE
    notes: |
      Partial fills are common in India (liquidity-driven) and US (market hours).
      Deterministic logic (matching by parent order) but needs tolerance for rounding,
      partial reversals, or fee adjustments per leg.
  severity:
    level: HIGH
    impact: Mismatches can mask actual breaks; partial reversal may be missed
    auto_escalate: true
  example_breaks:
    - "Order for 1000 shares: filled 600 on 10:00 AM, 400 on 10:30 AM. Bank shows only 60
    - "Partial cancellation: 100 shares cancelled out of 1000, but reversal cash not in b
    - "Broker reports 900 filled, 100 cancelled; but bank cash matches 1000 full qty."
  example_fix:
    - "Ops confirms with broker: is second leg still pending? Update trade status if canc
    - "Ops finds reversal is pending from broker (known delay); defer break aging."
    - "Ops reconciles: 100 cancelled = request not sent / not confirmed; broker confirms
  tags: [CORE, PHASE_1]
```

A3: Brokerage Fee Embedded in Net Amount

```
- id: TRADE_CASH_003
  domain: Trade ↔ Cash
  name: Broker Fee Decomposition and Reconciliation
  type: DETERMINISTIC
  tier: 1
  description: |
    When broker fees (brokerage, STT, exchange fees, GST) are embedded in net_amount
    (not listed separately), validate:
    (a) gross_amount - all_fees = net_amount (arithmetic check)
    (b) fee_amounts match broker's fee schedule or prior agreed rates
    (c) GST (if applicable) is 18% of eligible fees (India rule)
  inputs:
```

```
    - trade.gross_amount: quantity × price, before fees
    - trade.brokerage: commission %
    - trade.stt: STT amount (India: 0.1% on F&O, 0% on equities as of 2024; varies)
    - trade.exchange_fee: NSE/BSE fees
    - trade.gst: GST on brokerage and exchange fees
    - trade.net_amount: cash settlement amount
  logic_pseudocode: |
    expected_net = trade.gross_amount
                 - trade.brokerage
                 - trade.stt
                 - trade.exchange_fee
                 - trade.gst

    if side == SELL:  # In India, selling involves STT
      expected_stt = trade.gross_amount * 0.001  # 0.1% (as of 2024; SEBI may change)
    else:
      expected_stt = 0

    expected_gst = (trade.brokerage + trade.exchange_fee) * 0.18  # 18% in India

    if abs(trade.net_amount - expected_net) > tolerance_abs:
      flag FEE_MISMATCH

    if abs(trade.stt - expected_stt) > tolerance_abs:
      flag STT_MISMATCH (may be intentional override, or broker error)

    if abs(trade.gst - expected_gst) > tolerance_abs and trade.gst != 0:
      flag GST_MISMATCH
  suggested_tolerances:
    amount_abs: 0.5  # GST calculation may have rounding
    amount_pct: 0
    date_offset_days: 0
  region_applicability: INDIA (STT/GST specifics); GLOBAL (fee logic)
  rule_classification:
    tier: TIER_1_DETERMINISTIC
    should_live_in: RULE_ENGINE
    notes: |
      Core fee reconciliation. Deterministic because it's pure arithmetic.
      India-specific: STT rates change; rule is parameterized.
  severity:
    level: HIGH
    impact: Impacts fund P&L directly; wrong fee masks true PnL or hides broker overcharg
    auto_escalate: false (flag for review, not auto-escalate unless >threshold)
  example_breaks:
    - "Gross INR 1,00,000; brokerage 0.05% = INR 50; STT INR 100; exchange INR 10; GST sh
    - "Trade marked SELL, but STT is INR 0 (should be ~INR 100 per rate)."
    - "Net amount is INR 98,500 but calculated should be INR 98,490; INR 10 unaccounted."
  example_fix:
    - "Ops verifies fee schedule from broker for that period. If within approved schedule
    - "Ops checks: was STT waived or does this broker code have exemption? May be valid."
    - "Ops reconciles: rounding in GST calc; 1 rupee diff is acceptable, created journal
  tags: [CORE, PHASE_1, INDIA_SPECIFIC]
```

A4: Currency and Settlement Mismatch

```
- id: TRADE_CASH_004
  domain: Trade ↔ Cash
  name: Currency and Settlement Currency Validation
  type: DATA_QUALITY
  tier: 1
  description: |
    Validate that trade.settlement_currency == bank_txn.currency (or explicitly mapped).
    If FX forward is involved, ensure the forward contract is matched and rate is recorde
    Flag if trade booked in one currency but cash settled in another without a forward.
  inputs:
    - trade.trade_currency: currency trade was executed in (e.g., USD)
    - trade.settlement_currency: currency for cash settlement
    - trade.net_amount_trade_ccy: amount in trade currency
    - trade.net_amount_settlement_ccy: amount after any FX conversion
    - trade.fx_forward_id: reference to FX forward contract (if hedged)
    - trade.fx_forward_rate: locked rate (if applicable)
    - bank_txn.currency: currency of bank entry
    - bank_txn.amount: cash amount
  logic_pseudocode: |
    if trade.trade_currency == trade.settlement_currency:
      # Straightforward: no FX conversion
      if bank_txn.currency != trade.settlement_currency:
        flag CURRENCY_MISMATCH (trade says USD settle, bank shows INR)
    else:
      # FX conversion involved
      if trade.fx_forward_id is None or blank:
        flag MISSING_FX_FORWARD (trade says USD, settle in INR, but no forward ref)

      # Validate FX rate
      if trade.fx_forward_rate is not None:
        calculated_settle_amount = trade.net_amount_trade_ccy * trade.fx_forward_rate
        if abs(bank_txn.amount - calculated_settle_amount) > tolerance:
          flag FX_RATE_MISMATCH
  suggested_tolerances:
    amount_abs: 0
    amount_pct: 0.1  # FX rates may have small variance (0.1% acceptable)
    date_offset_days: 0
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_1_DETERMINISTIC
    should_live_in: RULE_ENGINE
    notes: Data quality gate; prevents downstream confusion.
  severity:
    level: CRITICAL
    impact: Wrong currency settlement is a settlement failure; impacts liquidity and reco
    auto_escalate: true
  example_breaks:
    - "Trade: BUY 100 AAPL in USD. Trade.settlement_currency = USD. But bank_txn.currency
    - "Trade: Trade in USD, settle in INR per forward contract. But trade.fx_forward_id i
    - "Trade: Forward rate locked at USD 1 = INR 83. Calculated settle = INR 15,050. But
  example_fix:
    - "Ops confirms with broker/custodian: was FX conversion done? If yes, create/link FX
    - "Ops: if forward was verbal or not in system, creates forward entry retroactively (
```

```
    - "Ops: FX rate variance is within market tolerance; create journal entry to track ga
  tags: [CORE, PHASE_1]
```

A5: Trade Status vs Cash Status Alignment

```
- id: TRADE_CASH_005
  domain: Trade ↔ Cash
  name: Trade and Cash Settlement Status Alignment
  type: DETERMINISTIC
  tier: 1
  description: |
    Enforce alignment: if trade.status = SETTLED, cash.status must also be SETTLED (or PO
    If trade.status = UNSETTLED, cash.status should be UNSETTLED or absent.
    If trade.status = CANCELLED, cash should show a reversal OR no entry at all (dependin
  inputs:
    - trade.trade_id
    - trade.status: UNSETTLED | SETTLED | CANCELLED | REJECTED
    - trade.settlement_date
    - cash.txn_id
    - cash.status: PENDING | POSTED | REVERSED | FAILED
    - cash.value_date
  logic_pseudocode: |
    for each trade:
      if trade.status == SETTLED:
        matching_cash = find_cash_entry(trade.id)
        if matching_cash is None:
          flag SETTLED_TRADE_NO_CASH
        elif matching_cash.status not in [POSTED, SETTLED, CONFIRMED]:
          flag CASH_NOT_POSTED_FOR_SETTLED_TRADE

      elif trade.status == UNSETTLED:
        matching_cash = find_cash_entry(trade.id)
        if matching_cash exists and matching_cash.status == POSTED:
          flag UNSETTLED_TRADE_HAS_POSTED_CASH

      elif trade.status == CANCELLED:
        matching_cash = find_cash_entry(trade.id)
        if matching_cash exists and matching_cash.status != REVERSED:
          # If cash was posted for a cancelled trade, should have reversal
          flag CANCELLED_TRADE_CASH_NOT_REVERSED
  suggested_tolerances:
    date_offset_days: 2  # Allow T+2 settlement lag
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_1_DETERMINISTIC
    should_live_in: RULE_ENGINE
    notes: Status alignment is a must-have consistency check.
  severity:
    level: CRITICAL
    impact: Status mismatch hides settlements that failed or cash that was double-posted
    auto_escalate: true
  example_breaks:
    - "Trade marked SETTLED, but bank_txn.status = PENDING (cash not yet posted)"
    - "Trade marked UNSETTLED, but cash already posted and balance affected"
```

```
    - "Trade marked CANCELLED, but original cash posting not reversed (balance error)"
  example_fix:
    - "Ops checks custodian/broker: is trade actually settled? Update trade status or fla
    - "Ops investigates cash posting: should not have happened for unsettled trade; creat
    - "Ops creates reversal entry for cancelled trade cash."
  tags: [CORE, PHASE_1]
```

A6: Cash Timing Variance (Expected vs Actual)

```
- id: TRADE_CASH_006
  domain: Trade ↔ Cash
  name: Settlement Date vs Value Date Variance
  type: TOLERANCE
  tier: 2
  description: |
    Track timing differences between expected settlement_date (T+1, T+2 per asset) and
    actual cash value_date. Expected variance: 0–2 days. Beyond that, flag as late settle
    Useful for identifying custodian delays, banking holidays, or settlement failures.
  inputs:
    - trade.settlement_date: expected date
    - trade.asset_class: determines standard settlement (T+1 vs T+2)
    - bank_txn.value_date: actual cash posting
  logic_pseudocode: |
    expected_settlement_date = trade.settlement_date
    actual_settlement_date = bank_txn.value_date

    days_late = (actual_settlement_date - expected_settlement_date).days

    if days_late < 0:
      flag EARLY_SETTLEMENT (unusual; may indicate trade date error or pre-settlement)
    elif days_late > 2:
      flag LATE_SETTLEMENT (investigate custodian delay or banking holiday)
    elif days_late in [0, 1, 2]:
      status = EXPECTED_VARIANCE (OK)
  suggested_tolerances:
    date_offset_days: 2
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_2_TOLERANCE
    should_live_in: RULE_ENGINE
    notes: Tolerance rule; helps ops diagnose settlement issues without false alarms.
  severity:
    level: MEDIUM
    impact: Late settlement may indicate custodian issue or require follow-up
    auto_escalate: false (age and monitor; escalate if >5 days)
  example_breaks:
    - "Trade settled for 2-Jan (T+1), but cash only on 5-Jan (3 days late)"
    - "Trade settled for 2-Jan, but cash on 31-Dec (pre-settlement? possible if trade dat
  example_fix:
    - "Ops checks calendar: are 3–4 Jan banking holidays? If so, expected. If not, raise
    - "Ops reviews trade_date; if incorrect, corrects and re-reconciles."
  tags: [CORE, PHASE_1]
```

A7–A20: Additional Trade ↔ Cash Rules

Continuing with the same structure, here are the remaining 14 rules (abbreviated for brevity):

```
- id: TRADE_CASH_007
  domain: Trade ↔ Cash
  name: Reversal and Cancellation Reconciliation
  type: EDGE_CASE
  tier: 2
  description: When a trade is cancelled or reversed, ensure the cash is also reversed (r
  inputs: [trade.status, trade.net_amount, bank_txn.status, bank_txn.amount]
  logic_pseudocode: |
    if trade.status == CANCELLED and (reversed_trade := find_reversal_trade(trade.id)):
      reversed_cash = find_cash_entry(reversed_trade.id)
      if reversed_cash.amount == -trade.net_amount:
        status = MATCHED_REVERSAL
      else:
        flag REVERSAL_AMOUNT_MISMATCH
  suggested_tolerances: {amount_abs: 0}
  region_applicability: GLOBAL
  severity: {level: HIGH, impact: "Reversed cash not undone = phantom settlement"}
  tags: [CORE, PHASE_1]

- id: TRADE_CASH_008
  domain: Trade ↔ Cash
  name: Buy vs Sell Side Cash Direction
  type: DATA_QUALITY
  tier: 1
  description: For BUY trades, cash must be outflow (debit). For SELL trades, cash must b
  inputs: [trade.side, trade.net_amount, bank_txn.amount]
  logic_pseudocode: |
    if trade.side == BUY and bank_txn.amount > 0:
      flag BUY_SELL_SIGN_MISMATCH
    elif trade.side == SELL and bank_txn.amount < 0:
      flag BUY_SELL_SIGN_MISMATCH
  suggested_tolerances: {amount_abs: 0}
  severity: {level: CRITICAL, impact: "Sign mismatch is data error, hides true cash posit
  tags: [CORE, PHASE_1]

- id: TRADE_CASH_009
  domain: Trade ↔ Cash
  name: Brokerage Fee Rate Validation
  type: DATA_QUALITY
  tier: 2
  description: Validate brokerage fee % against broker's approved rate sheet. Flag if out
  inputs: [trade.broker, trade.brokerage_amount, trade.gross_amount, broker_rate_sheet]
  logic_pseudocode: |
    approved_rate = broker_rate_sheet[trade.broker].max_rate
    actual_rate = trade.brokerage_amount / trade.gross_amount
    if actual_rate > approved_rate * 1.05:  # 5% flex
      flag BROKERAGE_OVERCHARGE
  suggested_tolerances: {amount_pct: 5}
  severity: {level: MEDIUM, impact: "Broker overcharge erodes fund returns"}
  tags: [PHASE_1]

- id: TRADE_CASH_010
```

```
  domain: Trade ↔ Cash
  name: Unmatched Buy-Sell Pairs (Same Security, Same Day)
  type: DETERMINISTIC
  tier: 1
  description: If same security bought and sold on same trade_date (intra-day trading), v
  inputs: [broker_trades[symbol, trade_date, side], bank_txns]
  logic_pseudocode: |
    for each symbol, date:
      buy_trades = filter(side=BUY, trade_date)
      sell_trades = filter(side=SELL, trade_date)
      if len(buy_trades) > 0 and len(sell_trades) > 0:
        total_buy_cash = sum(buy_trades.net_amount)
        total_sell_cash = sum(sell_trades.net_amount)
        if abs(total_buy_cash + total_sell_cash) > tolerance:
          flag INTRADAY_PAIR_NOT_BALANCED
  suggested_tolerances: {amount_abs: 100}
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: TRADE_CASH_011
  domain: Trade ↔ Cash
  name: Corporate Action Day: Dividend Cash In
  type: EDGE_CASE
  tier: 2
  description: |
    On ex-dividend date or payment date, verify:
    - Cash in matches expected dividend per holdings (quantity × dividend_per_share)
    - Timing: payment date +/- 1 day (custodian / bank delay common)
  inputs: [holdings[as_of_date, quantity], corp_actions[ex_date, dividend_per_share], ban
  logic_pseudocode: |
    for each ex_date in corp_actions where type == DIVIDEND:
      expected_dividend = holdings[as_of_date <= ex_date].quantity * corp_action.dividend
      matching_cash = bank_txns[(description contains symbol or isin) and (value_date ~ p
      if matching_cash.amount != expected_dividend:
        flag DIVIDEND_AMOUNT_MISMATCH
  suggested_tolerances: {amount_abs: 1, date_offset_days: 2}
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: TRADE_CASH_012
  domain: Trade ↔ Cash
  name: Multi-Leg Derivatives Settlement (Futures / Options)
  type: EDGE_CASE
  tier: 3
  description: For derivatives (futures, options), match margin calls, premium, and final
  inputs: [futures_trades, option_trades, margin_calls, bank_txns]
  logic_pseudocode: |
    for each derivative_trade:
      expected_cash = premium + margin_call_nets + final_pnl_if_expired
      matched_total = sum(bank_txns[trade_id])
      if abs(matched_total - expected_cash) > tolerance:
        flag DERIVATIVE_SETTLEMENT_MISMATCH
  suggested_tolerances: {amount_pct: 1}
  severity: {level: HIGH}
  tags: [OPTIONAL, PHASE_2]
```

```yaml
- id: TRADE_CASH_013
  domain: Trade ↔ Cash
  name: GST / Tax Variance on Fees
  type: DATA_QUALITY
  tier: 2
  description: Validate GST applied on broker/exchange fees is 18% (India) or applicable
  inputs: [trade.fees_breakdown, trade.gst_rate]
  logic_pseudocode: |
    eligible_fees = trade.brokerage + trade.exchange_fee  # NOT STT
    expected_gst = eligible_fees * 0.18
    if abs(trade.gst - expected_gst) > 0.5:
      flag GST_CALCULATION_ERROR
  suggested_tolerances: {amount_abs: 0.5}
  region_applicability: INDIA
  severity: {level: MEDIUM}
  tags: [INDIA_SPECIFIC, PHASE_1]

- id: TRADE_CASH_014
  domain: Trade ↔ Cash
  name: T+0 Same-Day Settlement Check
  type: DATA_QUALITY
  tier: 2
  description: For intra-day or same-day trades (T+0 settlement), verify cash is posted s
  inputs: [trade.trade_date, trade.settlement_date, bank_txn.value_date]
  logic_pseudocode: |
    if trade.settlement_date == trade.trade_date:  # T+0
      if bank_txn.value_date > trade.trade_date + 1:
        flag T0_LATE_SETTLEMENT
  suggested_tolerances: {date_offset_days: 1}
  region_applicability: GLOBAL
  severity: {level: MEDIUM}
  tags: [PHASE_2]

- id: TRADE_CASH_015
  domain: Trade ↔ Cash
  name: Broker Commission: Fixed vs. Percentage Reconciliation
  type: DETERMINISTIC
  tier: 1
  description: |
    Some brokers charge fixed commissions (e.g., INR 100 per trade), others percentage (e
    Validate which method applies per broker agreement and verify amount.
  inputs: [trade.broker, trade.brokerage_amount, trade.gross_amount, broker_agreement]
  logic_pseudocode: |
    broker_method = broker_agreement[trade.broker].commission_type  # FIXED or PERCENTAGE
    if broker_method == FIXED:
      expected_commission = broker_agreement.fixed_amount
    else:
      expected_commission = trade.gross_amount * broker_agreement.percentage_rate

    if abs(trade.brokerage_amount - expected_commission) > tolerance:
      flag BROKERAGE_MISMATCH
  suggested_tolerances: {amount_abs: 1, amount_pct: 0.5}
  severity: {level: MEDIUM}
  tags: [CORE, PHASE_1]

- id: TRADE_CASH_016
```

```
  domain: Trade ↔ Cash
  name: Contra Entry Detection (Off-Market Transfers)
  type: DATA_QUALITY
  tier: 2
  description: |
    Detect internal transfers (contra entries) between fund accounts. These are not marke
    Reconcile cash in one account to cash out of another account (same amount, same date)
  inputs: [bank_txns[account_id, amount, description], trade_source]
  logic_pseudocode: |
    for each bank_txn where description contains "transfer" or "contra":
      if not associated_trade_id:
        counterpart_txn = find_txn(
          opposite_amount,
          opposite_account,
          same_date
        )
        if counterpart_txn:
          status = CONTRA_MATCHED
        else:
          flag UNMATCHED_CONTRA
  suggested_tolerances: {amount_abs: 0, date_offset_days: 0}
  severity: {level: HIGH}
  tags: [OPTIONAL, PHASE_1]

- id: TRADE_CASH_017
  domain: Trade ↔ Cash
  name: Withholding Tax (TDS) on Interest / Dividends
  type: DATA_QUALITY
  tier: 2
  description: |
    For dividend/interest cash-in, if TDS (tax deducted at source) is applicable,
    verify: dividend_gross = dividend_net + tds_amount, and TDS rate is as per regulation
  inputs: [bank_txn[amount, description], corp_actions[dividend_per_share], tds_rate]
  logic_pseudocode: |
    if description contains "dividend" or "interest":
      tds_rate = applicable_rate_for_entity  # e.g., 20% for NRI, 0% for local
      expected_gross = bank_txn.amount / (1 - tds_rate)
      expected_tds = expected_gross * tds_rate
      # Verify against corp_action or tax register
  suggested_tolerances: {amount_pct: 1}
  region_applicability: INDIA
  severity: {level: MEDIUM}
  tags: [INDIA_SPECIFIC, PHASE_1]

- id: TRADE_CASH_018
  domain: Trade ↔ Cash
  name: Broker Netting: Gross Payable vs Net Settlement
  type: TOLERANCE
  tier: 2
  description: |
    Some brokers provide a "netting account" where multiple trades on a day are netted
    into a single cash settlement. Verify: sum of net_amounts = broker's net settlement ]
  inputs: [broker_trades[trade_date, net_amount], broker_settlement_report[net_amount]]
  logic_pseudocode: |
    for each broker, settlement_date:
      internal_net = sum(broker_trades[broker, settlement_date].net_amount)
```

```
      broker_reported_net = broker_settlement_report[broker, settlement_date].net
      if abs(internal_net - broker_reported_net) > tolerance:
        flag BROKER_NETTING_MISMATCH
  suggested_tolerances: {amount_abs: 1}
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: TRADE_CASH_019
  domain: Trade ↔ Cash
  name: Multi-Broker Account: Clearing Bank Reconciliation
  type: DETERMINISTIC
  tier: 1
  description: |
    If fund uses multiple brokers but single clearing bank account, ensure sum of all bro
    matches the bank's daily net cash position (for settlement date).
  inputs: [broker_trades[broker, settlement_date, net_amount], bank_txn[settlement_date,
  logic_pseudocode: |
    for each settlement_date:
      total_broker_net = sum(broker_trades[settlement_date].net_amount)
      bank_daily_net = bank_txn[settlement_date].net  # or sum of daily txns
      if abs(total_broker_net - bank_daily_net) > tolerance:
        flag MULTI_BROKER_SETTLEMENT_MISMATCH
  suggested_tolerances: {amount_abs: 10}  # Allow for rounding/intra-day movements
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: TRADE_CASH_020
  domain: Trade ↔ Cash
  name: Over-the-Counter (OTC) Trade Settlement Matching
  type: EDGE_CASE
  tier: 2
  description: |
    For OTC trades (not exchange-listed), no exchange settlement; cash settled bilateral]
    Match trade to bank entry with counterparty reference and amount.
  inputs: [otc_trades[counterparty, amount, settlement_date, trade_id], bank_txns[descrip
  logic_pseudocode: |
    for each otc_trade:
      matching_cash = bank_txns[
        amount ~= otc_trade.amount,
        description contains otc_trade.counterparty,
        value_date ~ settlement_date
      ]
      if matching_cash:
        status = MATCHED
      else:
        flag OTC_SETTLEMENT_MISSING
  suggested_tolerances: {amount_abs: 0.5, amount_pct: 0.1, date_offset_days: 3}
  severity: {level: HIGH}
  tags: [OPTIONAL, PHASE_2]
```

DOMAIN B: POSITIONS ↔ CUSTODIAN (15 Rules)

```
- id: POS_CUST_001
  domain: Positions ↔ Custodian
  name: Holdings Quantity Exact Match
  type: DETERMINISTIC
  tier: 1
  description: |
    For each security (isin), reconcile internal holdings quantity against custodian DP s
    Expected: exact match (1 unit = 1 unit). Mismatch indicates missed trade, reversal no
    or corporate action adjustment not applied.
  inputs:
    - internal_holdings.quantity: cumulative qty from trades + corporate actions
    - custodian_holdings.quantity: DP statement qty
    - internal_holdings.isin: security identifier
    - as_of_date: date of reconciliation
  logic_pseudocode: |
    for each as_of_date:
      for each isin:
        internal_qty = internal_holdings[isin, as_of_date].quantity
        custodian_qty = custodian_holdings[isin, as_of_date].quantity

        if internal_qty == custodian_qty:
          status = MATCHED
        else:
          qty_diff = custodian_qty - internal_qty
          if qty_diff > 0:
            flag MISSING_DEPOSIT (custodian has more, internal missing receipt)
          else:
            flag MISSING_WITHDRAWAL (internal has more, custodian missing debit)
  suggested_tolerances:
    qty_tolerance: 0  # Exact match expected
  region_applicability: GLOBAL
  rule_classification:
    tier: TIER_1_DETERMINISTIC
    should_live_in: RULE_ENGINE
  severity:
    level: CRITICAL
    impact: "AUM error; if missed qty is material, fund NAV is wrong"
  example_breaks:
    - "Internal: 1000 shares Apple. Custodian (DP): 950 shares. (50 shares missing deposi
    - "Internal: 500 shares TCS. Custodian: 510 shares. (10 shares unexpected; may be div
  example_fix:
    - "Ops investigates: check trade confirmations for recent buys (should match). If fou
    - "Ops checks: if 10 shares were reinvested dividend, creates matching entry and link
  tags: [CORE, PHASE_1, MUST_PASS]

- id: POS_CUST_002
  domain: Positions ↔ Custodian
  name: Corporate Action Quantity Adjustment
  type: DETERMINISTIC
  tier: 1
  description: |
    After corporate action (split, bonus, rights, etc.), holdings quantity must be adjust
    across both internal and custodian. Verify adjustment was applied.
  inputs:
```

```
    - corp_action.ex_date
    - corp_action.type: SPLIT | BONUS | RIGHTS
    - corp_action.ratio: (e.g., 5:1 for 5-way split)
    - holdings[as_of_date < ex_date].quantity: pre-CA qty
    - holdings[as_of_date >= ex_date].quantity: post-CA qty
    - custodian_holdings[as_of_date >= ex_date].quantity
  logic_pseudocode: |
    for each corp_action where type in [SPLIT, BONUS]:
      pre_ca_qty = internal_holdings[isin, ex_date - 1].quantity
      expected_post_ca_qty = pre_ca_qty * (corp_action.ratio_numerator / corp_action.rati

      actual_post_ca_qty = internal_holdings[isin, ex_date].quantity
      custodian_post_ca_qty = custodian_holdings[isin, ex_date].quantity

      if actual_post_ca_qty != expected_post_ca_qty:
        flag CA_ADJUSTMENT_NOT_APPLIED_INTERNAL

      if custodian_post_ca_qty != expected_post_ca_qty:
        flag CA_ADJUSTMENT_NOT_APPLIED_CUSTODIAN
  suggested_tolerances:
    qty_tolerance: 0
  region_applicability: GLOBAL
  severity: {level: CRITICAL, impact: "CA not applied = PnL wrong, cost basis wrong, AUM
  tags: [CORE, PHASE_1]

- id: POS_CUST_003
  domain: Positions ↔ Custodian
  name: Pledged vs Free Holdings Validation
  type: DATA_QUALITY
  tier: 2
  description: |
    For collateral management, validate: pledged_qty + free_qty = total_qty.
    Reconcile against custodian's pledge registry (if they maintain it).
  inputs:
    - holdings.quantity: total
    - holdings.pledged_qty: (if tracked internally)
    - holdings.free_qty: available for sale
    - custodian_pledge_register.pledged_qty
  logic_pseudocode: |
    for each holding:
      expected_total = pledged_qty + free_qty
      if holdings.quantity != expected_total:
        flag PLEDGED_FREE_MISMATCH

      # Optional: cross-check with custodian
      if custodian_pledge_register exists:
        if pledged_qty != custodian_pledge_register.pledged_qty:
          flag PLEDGE_MISMATCH_CUSTODIAN
  suggested_tolerances:
    qty_tolerance: 0
  region_applicability: GLOBAL (common in Indian AIF/mutual funds)
  severity: {level: HIGH, impact: "Incorrect free qty may allow sales that breach collate
  tags: [PHASE_1]

- id: POS_CUST_004
  domain: Positions ↔ Custodian
```

```
  name: Holdings Valuation (Market Price) Mismatch
  type: TOLERANCE
  tier: 2
  description: |
    Holdings are valued using market_price × quantity. If internal uses different price s
    (stale, from vendor A; custodian uses vendor B), values may differ. Validate variance
  inputs:
    - holdings.quantity
    - holdings.market_price: internal source
    - holdings.total_value: quantity × market_price
    - custodian_holdings.market_price: custodian source
    - custodian_holdings.total_value
  logic_pseudocode: |
    for each holding:
      internal_value = holdings.quantity * holdings.market_price
      custodian_value = custodian_holdings.quantity * custodian_holdings.market_price

      value_diff_abs = abs(internal_value - custodian_value)
      value_diff_pct = value_diff_abs / custodian_value

      if value_diff_pct > tolerance_pct:
        if (holdings.market_price - custodian_holdings.market_price).abs() > price_variar
          flag PRICE_SOURCE_MISMATCH
        else:
          flag VALUATION_VARIANCE
  suggested_tolerances:
    amount_pct: 2  # 2% valuation variance acceptable (price source difference)
  region_applicability: GLOBAL
  severity: {level: MEDIUM, impact: "Valuation variance affects AUM, not trade breaks"}
  tags: [PHASE_1]

- id: POS_CUST_005
  domain: Positions ↔ Custodian
  name: New Security (Not Yet in Custodian)
  type: EDGE_CASE
  tier: 2
  description: |
    Newly bought security: internal holds qty, but custodian DP may not yet show it (sett
    Allow grace period; flag if persists > T+2.
  inputs:
    - internal_holdings[trade_date, isin].quantity: >0
    - custodian_holdings[isin].quantity: 0 or missing
    - time_since_trade
  logic_pseudocode: |
    for each holding with internal_qty > 0 and custodian_qty == 0:
      if time_since_trade <= settlement_window (T+2):
        status = EXPECTED_SETTLEMENT_LAG
      else:
        flag SECURITY_NOT_IN_CUSTODIAN (late settlement or missing)
  suggested_tolerances:
    date_offset_days: 2
  severity: {level: HIGH, impact: "If custodian has not received, settlement may have fai
  tags: [CORE, PHASE_1]

- id: POS_CUST_006
  domain: Positions ↔ Custodian
```

```
  name: Unexpected Security in Custodian (No Internal Trade)
  type: EDGE_CASE
  tier: 2
  description: |
    Custodian holds security that internal book has no record of. May be dividend reinves
    gift, or data lag. Investigate.
  inputs:
    - custodian_holdings[isin].quantity: > 0
    - internal_holdings[isin].quantity: 0 or missing
  logic_pseudocode: |
    for each custodian_holding:
      if internal_holdings[isin] is None or internal_qty == 0:
        # Check for known reasons:
        if is_dividend_reinvestment(isin) or is_bonus_issue(isin):
          status = EXPECTED_CA_RECEIPT
        else:
          flag UNMATCHED_SECURITY_IN_CUSTODIAN
  suggested_tolerances:
    qty_tolerance: 0
  severity: {level: MEDIUM, impact: "May indicate missing trade record or unaccounted rec
  tags: [PHASE_1]

- id: POS_CUST_007
  domain: Positions ↔ Custodian
  name: Cost Basis Reconciliation
  type: DATA_QUALITY
  tier: 2
  description: |
    If internal and custodian both track cost basis, reconcile. Cost basis = sum(trade_qt
    Mismatch may indicate trade reversal not reflected or incorrect trade price entry.
  inputs:
    - internal_holdings.cost_base: cumulative cost of all shares held
    - custodian_holdings.cost_base: (if provided)
    - holdings.quantity
  logic_pseudocode: |
    expected_cost_base = sum(settled_trades[isin].quantity * settled_trades[isin].price)

    if internal_holdings.cost_base != expected_cost_base:
      flag INTERNAL_COST_BASIS_MISMATCH

    if custodian_holdings.cost_base and custodian_holdings.cost_base != expected_cost_bas
      flag CUSTODIAN_COST_BASIS_MISMATCH
  suggested_tolerances:
    amount_pct: 0.5  # Very small tolerance for rounding
  severity: {level: MEDIUM, impact: "Cost basis error affects realized PnL and tax report
  tags: [PHASE_1]

- id: POS_CUST_008
  domain: Positions ↔ Custodian
  name: Haircut / Valuation Adjustment
  type: DATA_QUALITY
  tier: 2
  description: |
    Some securities (illiquid, high-risk) are subject to haircuts in margin calculations
    Verify: if haircut % is applied, haircut_amount = total_value × haircut_pct.
  inputs:
```

```
    - holdings.total_value
    - holdings.haircut_pct
    - holdings.haircut_amount
  logic_pseudocode: |
    for each holding with haircut_pct > 0:
      expected_haircut = holdings.total_value * holdings.haircut_pct
      if abs(holdings.haircut_amount - expected_haircut) > 0.5:
        flag HAIRCUT_CALCULATION_ERROR
  suggested_tolerances:
    amount_abs: 0.5
  region_applicability: GLOBAL (common for hedge funds, AIFs with leverage)
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: POS_CUST_009
  domain: Positions ↔ Custodian
  name: Stale Position: No Recent Trade or Activity
  type: DETERMINISTIC
  tier: 2
  description: |
    Flag holdings with no trade activity and no corporate action for >90 days. May indica
    orphaned position or data error.
  inputs:
    - holdings.quantity: >0
    - last_trade_date or last_activity_date
    - current_date
  logic_pseudocode: |
    for each holding:
      days_since_activity = current_date - max(last_trade_date, last_ca_date)
      if days_since_activity > 90:
        flag STALE_POSITION (review for orphaned security, delisted stock, etc.)
  suggested_tolerances:
    date_offset_days: 90
  severity: {level: LOW, impact: "Audit/review item; may indicate delisted or forgotten h
  tags: [OPTIONAL, PHASE_2]

- id: POS_CUST_010
  domain: Positions ↔ Custodian
  name: Negative Holdings Validation
  type: DATA_QUALITY
  tier: 1
  description: |
    Holdings quantity must be >= 0 (short-selling not allowed in India; US/EU may allow).
    Flag if negative qty appears unexpectedly (data entry error or settlement issue).
  inputs:
    - holdings.quantity
    - fund_strategy: LONG_ONLY | LONG_SHORT | HEDGE_FUND
  logic_pseudocode: |
    for each holding:
      if fund_strategy == LONG_ONLY and holdings.quantity < 0:
        flag NEGATIVE_QUANTITY_LONG_ONLY_FUND (data error or settlement failure)
      elif holdings.quantity < -100:  # Even in long_short, very negative is suspicious
        flag EXTREME_SHORT_POSITION (review for intentionality)
  suggested_tolerances:
    qty_tolerance: 0
  region_applicability: GLOBAL (rules vary by fund type and region)
```

```
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: POS_CUST_011
  domain: Positions ↔ Custodian
  name: Restricted / Locked-In Shares
  type: DATA_QUALITY
  tier: 2
  description: |
    Some holdings are restricted (e.g., promoter shares locked-in, insider holdings, etc.
    If tracked, validate: restricted_qty + unrestricted_qty = total_qty.
  inputs:
    - holdings.quantity
    - holdings.restricted_qty
    - holdings.unrestricted_qty
    - restriction_reason
    - restriction_end_date
  logic_pseudocode: |
    for each holding with restriction:
      if current_date >= restriction_end_date:
        flag RESTRICTION_EXPIRED (update status)

      expected_total = restricted_qty + unrestricted_qty
      if expected_total != total_qty:
        flag RESTRICTED_UNRESTRICTED_MISMATCH
  suggested_tolerances:
    qty_tolerance: 0
  severity: {level: MEDIUM}
  tags: [INDIA_SPECIFIC, PHASE_1]

- id: POS_CUST_012
  domain: Positions ↔ Custodian
  name: Position Aging Report (Show Holdings Movement)
  type: DETERMINISTIC
  tier: 2
  description: |
    Generate position aging by last modification date. Helps identify stale vs active hol
  inputs:
    - holdings[all]
    - last_modified_date
  logic_pseudocode: |
    for each holding:
      days_old = current_date - last_modified_date
      bucket = "0-7 days" if days_old <= 7 else "8-30 days" else "30-90 days" else ">90 d
      count_and_value_per_bucket
  severity: {level: LOW, impact: "Reporting/analytics"}
  tags: [OPTIONAL, PHASE_1]

- id: POS_CUST_013
  domain: Positions ↔ Custodian
  name: Multi-Account Holdings (Same Security Across Accounts)
  type: DATA_QUALITY
  tier: 2
  description: |
    If fund operates multiple accounts (DP accounts, margin account, etc.),
    validate total holdings = sum across all accounts for same security.
```

```
  inputs:
    - holdings[account_id, isin].quantity
  logic_pseudocode: |
    for each isin:
      total_across_accounts = sum(holdings[isin, all_accounts].quantity)
      for each account:
        if holdings[account, isin] > total_across_accounts:
          flag HOLDING_EXCEEDS_TOTAL (data error)
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: POS_CUST_014
  domain: Positions ↔ Custodian
  name: Dividend Receivable Accrual
  type: TOLERANCE
  tier: 2
  description: |
    Between ex-date and payment date, dividends are accrued (receivable asset).
    Verify accrued dividend = quantity × declared_dividend_per_share.
  inputs:
    - holdings.quantity
    - corp_actions.dividend_per_share
    - corp_actions.ex_date
    - corp_actions.payment_date
    - accrued_dividend (balance sheet)
  logic_pseudocode: |
    for each ex_date <= current_date < payment_date:
      expected_accrued = holdings.quantity * dividend_per_share
      actual_accrued = accrued_dividend_on_books
      if abs(actual_accrued - expected_accrued) > tolerance:
        flag DIVIDEND_ACCRUAL_MISMATCH
  suggested_tolerances:
    amount_abs: 1
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: POS_CUST_015
  domain: Positions ↔ Custodian
  name: Trading Lot Tracking (FIFO vs Average Cost)
  type: DATA_QUALITY
  tier: 3
  description: |
    Some jurisdictions require tracking lots (FIFO, average cost) for tax purposes.
    If enabled, validate cost basis calculation aligns with chosen method.
  inputs:
    - trading_lots[purchase_date, quantity, cost_per_share]
    - cost_method: FIFO | LIFO | AVERAGE_COST
  logic_pseudocode: |
    # Complex; requires AI for nuanced reconciliation
    # Deterministic baseline: verify cost_method consistency
    if settled_trades.count > 100:
      # Delegate to AI for detailed lot matching
      pass
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]
```

DOMAIN C: NAV & VALUATION CHECKS (12 Rules)

(Condensed for brevity; follow same detailed structure as above)

```
- id: NAV_VAL_001
  domain: NAV & Valuation
  name: Holdings Sum vs Total AUM
  type: DETERMINISTIC
  tier: 1
  description: |
    AUM = sum(holdings.total_value) + cash_balance.
    Verify internal AUM calc matches sum of position values and cash.
  inputs:
    - holdings[all].total_value
    - cash_balance
    - nav_log.aum
  logic_pseudocode: |
    calculated_aum = sum(holdings.total_value) + cash_balance
    if abs(nav_log.aum - calculated_aum) > tolerance:
      flag AUM_MISMATCH
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_002
  domain: NAV & Valuation
  name: NAV Per Unit Calculation
  type: DETERMINISTIC
  tier: 1
  description: |
    NAV per unit = AUM / total_units outstanding.
    Validate formula and ensure units match fund structure.
  inputs:
    - nav_log.aum
    - fund_info.total_units
    - nav_log.nav_per_unit
  logic_pseudocode: |
    calculated_nav = nav_log.aum / fund_info.total_units
    if abs(calculated_nav - nav_log.nav_per_unit) > 0.01:
      flag NAV_CALCULATION_ERROR
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_003
  domain: NAV & Valuation
  name: Internal NAV vs Custodian NAV Variance
  type: TOLERANCE
  tier: 2
  description: |
    Compare internal NAV calc with custodian's official NAV.
    Variance is expected (price source, fee timing); validate < threshold.
  inputs:
    - nav_log[source=INTERNAL].nav_per_unit
    - nav_log[source=CUSTODIAN].nav_per_unit
  logic_pseudocode: |
    nav_variance_pct = abs(internal_nav - custodian_nav) / custodian_nav
    if nav_variance_pct > tolerance_pct:
```

```
      flag NAV_VARIANCE_EXCEEDS_THRESHOLD
  suggested_tolerances:
    amount_pct: 1  # 1% variance typically acceptable
  severity: {level: CRITICAL, impact: "Client NAV, regulatory reporting"}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_004
  domain: NAV & Valuation
  name: Accrued Interest Valuation
  type: TOLERANCE
  tier: 2
  description: |
    For bond holdings, accrued interest = par_value × coupon_rate × days_since_coupon / 3
  inputs:
    - holdings[bond].par_value
    - holdings[bond].coupon_rate
    - holdings[bond].last_coupon_date
  logic_pseudocode: |
    days_accrued = current_date - last_coupon_date
    expected_accrued = par * coupon_rate * (days_accrued / 365)
    actual_accrued = holdings.accrued_interest
    if abs(actual - expected) > tolerance:
      flag ACCRUED_INTEREST_MISMATCH
  suggested_tolerances:
    amount_pct: 0.5
  severity: {level: MEDIUM}
  tags: [PHASE_2]

- id: NAV_VAL_005
  domain: NAV & Valuation
  name: Stale Price Check
  type: DATA_QUALITY
  tier: 2
  description: |
    Identify holdings priced using data >1 day old (stale price).
    Flag for manual review; especially critical for illiquid securities.
  inputs:
    - holdings.market_price
    - holdings.price_as_of_date
    - current_date
  logic_pseudocode: |
    for each holding:
      price_age = current_date - price_as_of_date
      if price_age > 1 and holding.is_illiquid:
        flag STALE_PRICE_ILLIQUID
  suggested_tolerances:
    date_offset_days: 1
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: NAV_VAL_006
  domain: NAV & Valuation
  name: Corporate Action Impact on NAV
  type: EDGE_CASE
  tier: 1
  description: |
```

```
    On ex-date of corporate action (split, bonus, dividend), NAV should adjust according]
    Validate NAV calc reflects CA impact.
  inputs:
    - holdings[pre_ex_date]
    - holdings[post_ex_date]
    - corp_actions.ex_date
    - nav_log[around ex_date]
  logic_pseudocode: |
    # Complex; typically handled post-CA adjustment
    # Verify NAV difference = CA impact (qty change + dividend payment)
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_007
  domain: NAV & Valuation
  name: Management Fee Accrual
  type: DETERMINISTIC
  tier: 1
  description: |
    Management fees are accrued daily or monthly. Validate: accrued_mgt_fee = AUM × fee_r
  inputs:
    - nav_log.aum
    - fund_fee_rate
    - last_fee_payment_date
  logic_pseudocode: |
    days_since_fee_payment = current_date - last_fee_payment_date
    expected_accrued = aum * fee_rate * (days / 365)
    actual_accrued = fee_accrual_on_books
    if abs(actual - expected) > tolerance:
      flag MGT_FEE_ACCRUAL_MISMATCH
  severity: {level: HIGH, impact: "Fund P&L"}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_008
  domain: NAV & Valuation
  name: Performance Fee Calculation
  type: DETERMINISTIC
  tier: 2
  description: |
    Performance fees (if applicable) = max(0, (current_nav - high_water_mark) × perf_fee_
  inputs:
    - nav_log.nav_per_unit
    - historical_nav.high_water_mark
    - fund_perf_fee_rate
  logic_pseudocode: |
    outperformance = max(0, nav - hwm)
    expected_perf_fee = outperformance * perf_fee_rate
    if abs(actual_perf_fee - expected_perf_fee) > tolerance:
      flag PERF_FEE_CALCULATION_ERROR
  suggested_tolerances:
    amount_pct: 0.1
  severity: {level: MEDIUM}
  tags: [PHASE_2]

- id: NAV_VAL_009
  domain: NAV & Valuation
```

```
  name: Fair Value Adjustment Tracking
  type: DATA_QUALITY
  tier: 2
  description: |
    For illiquid / non-standard securities, fair value adjustments may be needed.
    Validate: adjusted_value = market_price ± fv_adjustment.
  inputs:
    - holdings.market_price
    - holdings.fv_adjustment
    - holdings.fair_value
  logic_pseudocode: |
    for each holding with fv_adjustment != 0:
      expected_fv = market_price + fv_adjustment
      if abs(holdings.fair_value - expected_fv) > 0.01:
        flag FV_ADJUSTMENT_MISMATCH
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: NAV_VAL_010
  domain: NAV & Valuation
  name: Batch NAV Closure (End-of-Day Lock)
  type: DETERMINISTIC
  tier: 1
  description: |
    At EOD, NAV calculation must be locked (no further changes).
    Validate NAV is computed once, published, and not changed retroactively.
  inputs:
    - nav_log[date].status: DRAFT | PUBLISHED | LOCKED
    - nav_log[date].modification_count
  logic_pseudocode: |
    if nav_log.status == LOCKED and nav_log.modification_count > 1:
      flag LOCKED_NAV_WAS_MODIFIED (audit issue)
  severity: {level: HIGH, impact: "Regulatory, investor trust"}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_011
  domain: NAV & Valuation
  name: Negative NAV Detection
  type: DATA_QUALITY
  tier: 1
  description: |
    NAV per unit should never be negative (unless negative AUM, which is error).
  inputs:
    - nav_log.nav_per_unit
  logic_pseudocode: |
    if nav_per_unit < 0:
      flag NEGATIVE_NAV (critical data error)
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: NAV_VAL_012
  domain: NAV & Valuation
  name: Cross-Currency NAV Consistency
  type: TOLERANCE
  tier: 2
  description: |
```

```
    If fund has multi-currency holdings, NAV in base currency must equal sum of
    holdings in each currency converted at agreed FX rate.
  inputs:
    - holdings[ccy].total_value
    - fx_rates[ccy_to_base]
    - nav_log.aum
  logic_pseudocode: |
    aum_base = sum(holdings[ccy].total_value * fx_rates[ccy_to_base])
    if abs(aum_base - nav_log.aum) > tolerance:
      flag MULTI_CURRENCY_NAV_MISMATCH
  suggested_tolerances:
    amount_pct: 1
  severity: {level: MEDIUM}
  tags: [PHASE_2]
```

DOMAIN D: FEES & CHARGES (10 Rules)

```
- id: FEE_CHG_001
  domain: Fees & Charges
  name: Brokerage Fee Total Reconciliation
  type: DETERMINISTIC
  tier: 1
  description: |
    Sum of all brokerage fees in broker_fees table should match broker's monthly bill.
  inputs:
    - broker_fees[broker, month].sum(amount) where fee_type == BROKERAGE
    - broker_bill[broker, month].brokerage_total
  logic_pseudocode: |
    internal_total = sum(broker_fees[broker, month] where type == BROKERAGE)
    broker_bill_total = broker_bill[broker, month].brokerage
    if abs(internal_total - broker_bill_total) > tolerance:
      flag BROKERAGE_FEE_MISMATCH
  suggested_tolerances:
    amount_abs: 10  # Allow rounding
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: FEE_CHG_002
  domain: Fees & Charges
  name: STT / Securities Transaction Tax (India)
  type: DATA_QUALITY
  tier: 1
  description: |
    STT is applicable on selling of equities in India (0.1% as of 2024; SEBI may change).
    Validate STT is applied only on SELL trades and at correct rate.
  inputs:
    - broker_fees[trade_id].fee_type == STT
    - broker_trades[trade_id].side
    - broker_fees.amount
    - broker_trades.gross_amount
  logic_pseudocode: |
    for each stt_fee:
      associated_trade = broker_trades[trade_id]
      if associated_trade.side != SELL:
```

```
        flag STT_APPLIED_ON_BUY (incorrect)

      expected_stt = associated_trade.gross_amount * 0.001
      if abs(stt_fee.amount - expected_stt) > tolerance:
        flag STT_RATE_MISMATCH
  suggested_tolerances:
    amount_abs: 0.5
  region_applicability: INDIA
  severity: {level: HIGH}
  tags: [INDIA_SPECIFIC, PHASE_1]

- id: FEE_CHG_003
  domain: Fees & Charges
  name: GST on Fees (India)
  type: DATA_QUALITY
  tier: 1
  description: |
    GST (18% in India as of 2024) is applied on brokerage and exchange fees, NOT on STT.
    Validate GST calculation: gst_amount = (brokerage + exchange_fee) × 0.18.
  inputs:
    - broker_fees[broker, trade_id].brokerage
    - broker_fees[broker, trade_id].exchange_fee
    - broker_fees[broker, trade_id].gst
  logic_pseudocode: |
    for each trade:
      eligible_for_gst = brokerage + exchange_fee  # NOT STT
      expected_gst = eligible_for_gst * 0.18
      if abs(broker_fees.gst - expected_gst) > tolerance:
        flag GST_CALCULATION_ERROR
  suggested_tolerances:
    amount_abs: 0.5
  region_applicability: INDIA
  severity: {level: MEDIUM}
  tags: [INDIA_SPECIFIC, PHASE_1]

- id: FEE_CHG_004
  domain: Fees & Charges
  name: Custody Fee Monthly Reconciliation
  type: TOLERANCE
  tier: 2
  description: |
    Custodian charges monthly custody fees (often AUM-based or per-transaction).
    Reconcile custodian invoice against internal fee register.
  inputs:
    - custody_fees[month].internal_total
    - custodian_invoice[month].custody_fee_line
  logic_pseudocode: |
    internal_custody_total = sum(custody_fees[month])
    invoice_custody_total = custodian_invoice[month].custody_fee
    if abs(internal - invoice) > tolerance:
      flag CUSTODY_FEE_MISMATCH
  suggested_tolerances:
    amount_abs: 100  # Allow for rounding, true-up
  severity: {level: MEDIUM}
  tags: [PHASE_1]
```

```
- id: FEE_CHG_005
  domain: Fees & Charges
  name: Exchange Fees Validation
  type: DATA_QUALITY
  tier: 2
  description: |
    Exchange fees (NSE/BSE, DTCC, etc.) vary by volume/asset type. Validate rates match e
  inputs:
    - broker_fees[trade_id].exchange_fee
    - broker_trades[trade_id].gross_amount
    - exchange_fee_schedule[exchange]
  logic_pseudocode: |
    for each trade:
      exchange_name = broker_trades[trade_id].exchange
      expected_fee_rate = exchange_fee_schedule[exchange_name]
      expected_fee = trade.gross_amount * expected_fee_rate
      if abs(broker_fees.exchange_fee - expected_fee) > tolerance:
        flag EXCHANGE_FEE_MISMATCH
  suggested_tolerances:
    amount_pct: 5  # Some brokers bundle or discount
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FEE_CHG_006
  domain: Fees & Charges
  name: Fee Timing: Accrual vs. Payment
  type: TOLERANCE
  tier: 2
  description: |
    Fees may be accrued daily but paid monthly. Validate accrued fees match
    (cumulative daily) and payment timing aligns with invoice.
  inputs:
    - fee_accrual[day].amount
    - custodian_invoice[month].fee_amount
    - sum(fee_accrual[days in month])
  logic_pseudocode: |
    monthly_accrued = sum(fee_accrual[all days in month])
    invoice_amount = custodian_invoice[month]
    if abs(monthly_accrued - invoice_amount) > tolerance:
      flag FEE_ACCRUAL_PAYMENT_MISMATCH
  suggested_tolerances:
    amount_abs: 10
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FEE_CHG_007
  domain: Fees & Charges
  name: Waived / Discounted Fees Approval
  type: DATA_QUALITY
  tier: 2
  description: |
    If fees are waived or discounted, ensure there's approval documentation.
    Flag unapproved waivers.
  inputs:
    - broker_fees[trade_id].original_amount
    - broker_fees[trade_id].actual_amount
```

```
    - broker_fees[trade_id].waiver_approval_flag
  logic_pseudocode: |
    for each fee where actual_amount < original_amount:
      if waiver_approval_flag != APPROVED:
        flag UNAPPROVED_WAIVER
  severity: {level: HIGH, impact: "Compliance risk"}
  tags: [PHASE_1]

- id: FEE_CHG_008
  domain: Fees & Charges
  name: Zero Fee Anomaly Detection
  type: DATA_QUALITY
  tier: 2
  description: |
    Flag trades with zero brokerage when expected. May indicate broker error or special c
  inputs:
    - broker_fees[trade_id].brokerage
    - broker_trades[trade_id].amount
  logic_pseudocode: |
    for each trade where brokerage == 0:
      if trade.amount > threshold and trade.broker != [known_zero_fee_broker]:
        flag ZERO_BROKERAGE_ANOMALY
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FEE_CHG_009
  domain: Fees & Charges
  name: Fee Reversal (If Trade is Cancelled)
  type: DETERMINISTIC
  tier: 1
  description: |
    If a trade is cancelled, associated fees should also be reversed (negative fee entry)
  inputs:
    - broker_trades[trade_id].status == CANCELLED
    - broker_fees[trade_id].amount
  logic_philippocode: |
    for each cancelled_trade:
      original_fee = broker_fees[trade_id]
      if original_fee exists:
        reversal_fee = find_fee_entry(trade_id, is_reversal=True)
        if reversal_fee.amount != -original_fee.amount:
          flag FEE_NOT_REVERSED_ON_TRADE_CANCEL
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: FEE_CHG_010
  domain: Fees & Charges
  name: CIF Charge (Custodian Integration Fee) Reconciliation
  type: TOLERANCE
  tier: 2
  description: |
    Some custodians charge CIF (Integration Fee). Reconcile monthly CIF bill vs. internal
  inputs:
    - custodian_invoice[month].cif_charge
    - cif_accrual[month]
  logic_pseudocode: |
```

```
    if abs(custodian_invoice.cif - cif_accrual) > tolerance:
      flag CIF_MISMATCH
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_1]
```

DOMAIN E: CORPORATE ACTIONS (12 Rules)

```
- id: CA_001
  domain: Corporate Actions
  name: Split Adjustment Validation
  type: DETERMINISTIC
  tier: 1
  description: |
    On split ex-date, holdings qty should be: old_qty × (split_ratio_numerator / split_ra
    Validate internal and custodian holdings both reflect split.
  inputs:
    - corp_actions[ex_date].type == SPLIT
    - corp_actions.ratio_numerator
    - corp_actions.ratio_denominator
    - holdings[ex_date - 1].quantity
    - holdings[ex_date].quantity
  logic_pseudocode: |
    split_ratio = ratio_numerator / ratio_denominator
    pre_split_qty = holdings[ex_date - 1].quantity
    expected_post_split = pre_split_qty * split_ratio
    actual_post_split = holdings[ex_date].quantity
    if abs(actual - expected) > 1:
      flag SPLIT_ADJUSTMENT_FAILED
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: CA_002
  domain: Corporate Actions
  name: Bonus Issue Quantity Increase
  type: DETERMINISTIC
  tier: 1
  description: |
    On bonus ex-date, qty increases: new_qty = old_qty × (1 + bonus_ratio).
  inputs:
    - corp_actions[ex_date].type == BONUS
    - corp_actions.ratio_numerator  (e.g., 1 for 5:1 bonus = 1 new share per 5 held)
    - corp_actions.ratio_denominator
    - holdings[ex_date - 1].quantity
    - holdings[ex_date].quantity
  logic_pseudocode: |
    bonus_ratio = ratio_numerator / ratio_denominator
    new_qty = old_qty * (1 + bonus_ratio)
    if abs(holdings[ex_date].quantity - new_qty) > 1:
      flag BONUS_ADJUSTMENT_FAILED
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: CA_003
  domain: Corporate Actions
```

```
  name: Cost Basis Adjustment for Splits/Bonus
  type: DETERMINISTIC
  tier: 1
  description: |
    For splits: adjusted_cost_per_share = original_cost_per_share / split_ratio.
    For bonus: cost basis unchanged (bonus cost = 0; only original shares get cost).
    Validate cost basis adjustments.
  inputs:
    - holdings[ex_date - 1].cost_per_share
    - holdings[ex_date].cost_per_share
    - corp_actions[ex_date].type
    - corp_actions.ratio
  logic_pseudocode: |
    if corp_action.type == SPLIT:
      adjusted_cost = original_cost / split_ratio
      if abs(holdings[ex_date].cost_per_share - adjusted_cost) > 0.01:
        flag SPLIT_COST_BASIS_NOT_ADJUSTED
    elif corp_action.type == BONUS:
      # Bonus shares have zero cost basis (tax treatment)
      # Only original shares retain cost
      bonus_qty = holdings[ex_date].quantity - holdings[ex_date - 1].quantity
      bonus_cost_base = bonus_qty * 0  # Should be zero
      if abs(bonus_cost_base) > 0.01:
        flag BONUS_COST_BASIS_WRONG
  severity: {level: CRITICAL, impact: "Tax/PnL reporting"}
  tags: [CORE, PHASE_1]

- id: CA_004
  domain: Corporate Actions
  name: Dividend Cash Reconciliation
  type: TOLERANCE
  tier: 2
  description: |
    On dividend payment date, cash received should equal: holdings_qty_on_ex_date × divid
  inputs:
    - holdings[ex_date].quantity
    - corp_actions.dividend_per_share
    - bank_txns[payment_date].amount
  logic_pseudocode: |
    expected_dividend = holdings[ex_date].quantity * dividend_per_share
    actual_dividend = bank_txns[payment_date].amount
    if abs(actual - expected) > tolerance:
      flag DIVIDEND_AMOUNT_MISMATCH
  suggested_tolerances:
    amount_abs: 1
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: CA_005
  domain: Corporate Actions
  name: Ex-Date Timing Validation
  type: DATA_QUALITY
  tier: 1
  description: |
    Ex-date marks the cutoff for CA eligibility. Ensure: ex_date < record_date < payment_
  inputs:
```

```
    - corp_actions.ex_date
    - corp_actions.record_date
    - corp_actions.payment_date
  logic_pseudocode: |
    if not (ex_date < record_date <= payment_date):
      flag CA_DATE_SEQUENCE_ERROR
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: CA_006
  domain: Corporate Actions
  name: Unprocessed CA Impact (Holdings Not Adjusted)
  type: EDGE_CASE
  tier: 2
  description: |
    If ex_date has passed but holdings not adjusted, flag for manual processing.
  inputs:
    - corp_actions[ex_date <= today]
    - holdings[ex_date].quantity
    - holdings[ex_date - 1].quantity
  logic_pseudocode: |
    for each ex_date <= today:
      if corp_action.type in [SPLIT, BONUS] and holdings[ex_date] == holdings[ex_date - 1
        flag CA_NOT_PROCESSED (qty not adjusted)
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: CA_007
  domain: Corporate Actions
  name: Rights Issue Allocation Tracking
  type: EDGE_CASE
  tier: 3
  description: |
    Rights issues offer new shares at a set ratio. Validate: rights_allotted = holdings ×
    and new shares received after record-date match allotment.
  inputs:
    - corp_actions[type == RIGHTS].rights_ratio
    - holdings[ex_date].quantity
    - broker_trades[post_record_date].quantity for rights_isin
  logic_pseudocode: |
    expected_rights_allotted = holdings[ex_date] * rights_ratio
    rights_received = broker_trades[record_date to settlement].quantity
    if abs(expected - received) > tolerance:
      flag RIGHTS_ALLOTMENT_MISMATCH
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: CA_008
  domain: Corporate Actions
  name: Dividend Reinvestment (DRIP) Detection
  type: EDGE_CASE
  tier: 2
  description: |
    Some funds have dividend reinvestment (DRIP) option. If enabled, expect:
    New units = dividend_cash / nav_on_payment_date (auto-purchase).
  inputs:
```

```
    - corp_actions[type == DIVIDEND].dividend_cash
    - nav_log[payment_date].nav_per_unit
    - holdings[post_payment_date].quantity
  logic_pseudocode: |
    if fund.drip_enabled:
      new_units_expected = dividend_cash / nav_per_unit
      actual_qty_increase = holdings[post] - holdings[pre]
      if abs(actual_qty_increase - new_units_expected) > 1:
        flag DRIP_UNITS_MISMATCH
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: CA_009
  domain: Corporate Actions
  name: Rights Lapsed (Unexercised Rights)
  type: EDGE_CASE
  tier: 3
  description: |
    Rights that are not exercised before expiry date lapse. Ensure lapsed rights are remo
  inputs:
    - corp_actions[type == RIGHTS].expiry_date
    - holdings[rights_isin].quantity
  logic_pseudocode: |
    for each rights_issue:
      if current_date > expiry_date and holdings[rights_isin].quantity > 0:
        flag LAPSED_RIGHTS_STILL_IN_HOLDINGS
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: CA_010
  domain: Corporate Actions
  name: Merger / Demerger Security Mapping
  type: EDGE_CASE
  tier: 3
  description: |
    On merger, old security replaced by new security (new isin). Ensure:
    Old holdings zeroed out, new holdings created with adjusted qty.
  inputs:
    - corp_actions[type == MERGER]
    - corp_actions.old_isin
    - corp_actions.new_isin
    - corp_actions.exchange_ratio
    - holdings[old_isin].quantity
    - holdings[new_isin].quantity
  logic_pseudocode: |
    old_qty = holdings[old_isin].quantity
    expected_new_qty = old_qty * exchange_ratio
    actual_new_qty = holdings[new_isin].quantity

    if old_qty > 0:
      flag OLD_SECURITY_NOT_ZEROED (should be zero post-merger)
    if abs(actual_new_qty - expected_new_qty) > 1:
      flag NEW_SECURITY_QTY_MISMATCH
  severity: {level: CRITICAL}
  tags: [OPTIONAL, PHASE_2]
```

```
- id: CA_011
  domain: Corporate Actions
  name: Tax Implications of CA (Tracking for Compliance)
  type: DATA_QUALITY
  tier: 3
  description: |
    Corporate actions have tax implications. Log ca_event with tax_treatment for audit tr
  inputs:
    - corp_actions[ca_id].type
    - corp_actions.tax_treatment (TAXABLE | NON_TAXABLE | DEFERRED)
  logic_pseudocode: |
    for each ca_event:
      if tax_treatment is None or blank:
        flag CA_TAX_TREATMENT_NOT_DOCUMENTED
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_1]

- id: CA_012
  domain: Corporate Actions
  name: Reverse Split (Negative Ratio)
  type: DETERMINISTIC
  tier: 1
  description: |
    Reverse splits (consolidations) have ratio < 1. Validate:
    new_qty = old_qty / reverse_split_ratio (e.g., 1:10 reverse split → qty / 10).
  inputs:
    - corp_actions[type == SPLIT].ratio_numerator
    - corp_actions.ratio_denominator
    - holdings[pre].quantity
    - holdings[post].quantity
  logic_pseudocode: |
    if ratio_numerator < ratio_denominator:  # Reverse split
      expected_new_qty = holdings[pre].quantity / (ratio_denominator / ratio_numerator)
    else:  # Normal split
      expected_new_qty = holdings[pre].quantity * (ratio_numerator / ratio_denominator)

    if abs(holdings[post].quantity - expected_new_qty) > 1:
      flag REVERSE_SPLIT_ADJUSTMENT_FAILED
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]
```

DOMAIN F: FX & MULTI-CURRENCY (12 Rules)

```
- id: FX_001
  domain: FX & Multi-Currency
  name: FX Trade Settlement Amount Validation
  type: TOLERANCE
  tier: 2
  description: |
    For trades in non-base currency, validate settlement cash = trade_amount × fx_rate.
  inputs:
    - trade.trade_currency
    - trade.net_amount_trade_ccy
    - trade.settlement_currency
```

```
    - trade.fx_rate_used
    - bank_txn[settlement].amount_settlement_ccy
  logic_pseudocode: |
    if trade.trade_currency != trade.settlement_currency:
      expected_settlement = trade.net_amount_trade_ccy * trade.fx_rate_used
      actual_settlement = bank_txn.amount_settlement_ccy
      if abs(actual - expected) > tolerance:
        flag FX_SETTLEMENT_AMOUNT_MISMATCH
  suggested_tolerances:
    amount_pct: 0.1  # FX spreads typically <0.1%
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: FX_002
  domain: FX & Multi-Currency
  name: FX Rate Sourcing Consistency
  type: DATA_QUALITY
  tier: 1
  description: |
    All FX conversions should use consistent rate source (custodian, market feed, etc.).
    Flag if mixed sources.
  inputs:
    - trade.fx_rate_source: CUSTODIAN | MARKET | DEAL
    - trade.fx_rate_value
  logic_pseudocode: |
    for each trade with fx_conversion:
      if trade.fx_rate_source not in [approved_sources]:
        flag UNKNOWN_FX_RATE_SOURCE
      # Optionally validate rate within market range
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FX_003
  domain: FX & Multi-Currency
  name: FX Forward Contract Matching
  type: EDGE_CASE
  tier: 2
  description: |
    If trade is hedged with FX forward, forward contract must exist and be matched.
  inputs:
    - trade.fx_forward_id
    - fx_forward_contracts[forward_id]
    - trade.net_amount_trade_ccy
    - forward.notional_amount
  logic_pseudocode: |
    if trade.fx_forward_id is not None:
      forward = fx_forward_contracts[trade.fx_forward_id]
      if forward is None:
        flag FORWARD_CONTRACT_NOT_FOUND
      elif forward.notional_amount != trade.net_amount_trade_ccy:
        flag FORWARD_NOTIONAL_MISMATCH
      elif forward.settlement_date != trade.settlement_date:
        flag FORWARD_SETTLEMENT_DATE_MISMATCH
  severity: {level: HIGH}
  tags: [PHASE_1]
```

```yaml
- id: FX_004
  domain: FX & Multi-Currency
  name: Realized FX Gain / Loss Tracking
  type: DETERMINISTIC
  tier: 1
  description: |
    When FX conversion occurs (deal rate vs spot), gain/loss is realized.
    Verify: fx_gain_loss = trade_amount × (spot_rate - deal_rate).
  inputs:
    - trade.net_amount_trade_ccy
    - trade.deal_rate
    - trade.spot_rate_at_time
    - trade.fx_gain_loss
  logic_pseudocode: |
    expected_fxpl = trade.net_amount_trade_ccy * (trade.spot_rate - trade.deal_rate)
    if abs(trade.fxpl - expected_fxpl) > tolerance:
      flag FX_PL_CALCULATION_MISMATCH
  suggested_tolerances:
    amount_abs: 1
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FX_005
  domain: FX & Multi-Currency
  name: Unrealized FX Gain / Loss (Mark-to-Market)
  type: TOLERANCE
  tier: 2
  description: |
    Holdings in foreign currency are revalued daily at spot rate (mark-to-market).
    Unrealized FX P&L = (current_spot - original_rate) × quantity.
  inputs:
    - holdings[non_base_ccy].quantity
    - holdings.original_fx_rate
    - fx_rates[current_spot]
    - holdings.unrealized_fxpl
  logic_pseudocode: |
    for each holding in non_base_currency:
      spot_rate_today = fx_rates[current_spot]
      expected_unrealized_fxpl = holding.quantity * (spot_rate_today - original_rate)
      if abs(holding.unrealized_fxpl - expected_unrealized_fxpl) > tolerance:
        flag UNREALIZED_FXPL_MISMATCH
  suggested_tolerances:
    amount_pct: 1
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: FX_006
  domain: FX & Multi-Currency
  name: Multi-Currency AUM Consolidation
  type: DETERMINISTIC
  tier: 1
  description: |
    Total AUM = sum(AUM_in_each_ccy_converted_to_base_ccy).
    Validate base currency conversion uses agreed rates.
  inputs:
    - holdings[ccy].total_value
```

```
    - fx_rates[ccy_to_base]
    - nav_log.aum_base_ccy
  logic_pseudocode: |
    aum_base = sum(holdings[ccy].total_value * fx_rates[ccy_to_base])
    if abs(nav_log.aum - aum_base) > tolerance:
      flag MULTI_CURRENCY_AUM_MISMATCH
  suggested_tolerances:
    amount_pct: 0.5
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: FX_007
  domain: FX & Multi-Currency
  name: FX Rounding Tolerance
  type: TOLERANCE
  tier: 2
  description: |
    FX conversions often have minor rounding differences. Tolerance for conversion:
    abs(calculated - actual) <= tolerance (e.g., 0.01 rupees).
  inputs:
    - trade.net_amount_trade_ccy
    - trade.fx_rate
    - trade.net_amount_settlement_ccy
  logic_pseudocode: |
    calculated_settlement = trade_amount * fx_rate
    actual_settlement = trade.net_amount_settlement_ccy
    rounding_diff = abs(calculated_settlement - actual_settlement)
    if rounding_diff > tolerance:
      flag FX_ROUNDING_ERROR
  suggested_tolerances:
    amount_abs: 0.5  # Half a unit acceptable
  severity: {level: LOW}
  tags: [PHASE_1]

- id: FX_008
  domain: FX & Multi-Currency
  name: Cross-Currency Pair Validation
  type: DATA_QUALITY
  tier: 1
  description: |
    Ensure FX rate is defined for the specific currency pair. Flag if rate missing for a
  inputs:
    - trade.trade_currency
    - trade.settlement_currency
    - fx_rates[trade_ccy_to_settle_ccy]
  logic_pseudocode: |
    for each trade with multi_ccy:
      pair = (trade.trade_currency, trade.settlement_currency)
      if fx_rates[pair] is None:
        flag FX_RATE_NOT_DEFINED_FOR_PAIR
  severity: {level: CRITICAL}
  tags: [PHASE_1]

- id: FX_009
  domain: FX & Multi-Currency
  name: Custodian FX Rate vs Market Rate
```

```
  type: TOLERANCE
  tier: 2
  description: |
    Custodian may apply FX rates different from market (spreads, markups).
    Flag if variance exceeds tolerance.
  inputs:
    - custodian_invoice.fx_rate_applied
    - market_fx_rate[date]
  logic_pseudocode: |
    rate_variance = abs(custodian_rate - market_rate) / market_rate
    if rate_variance > tolerance_pct:
      flag CUSTODIAN_FX_SPREAD_HIGH
  suggested_tolerances:
    amount_pct: 0.5  # Typical custodian spread
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_2]

- id: FX_010
  domain: FX & Multi-Currency
  name: Currency Mismatch on Trade vs Settlement
  type: DATA_QUALITY
  tier: 1
  description: |
    If trade currency is specified, settlement currency must align or FX forward must exi
  inputs:
    - trade.trade_currency
    - trade.settlement_currency
    - trade.fx_forward_id
  logic_pseudocode: |
    if trade.trade_currency != trade.settlement_currency:
      if trade.fx_forward_id is None:
        flag MULTI_CURRENCY_TRADE_NO_FORWARD
  severity: {level: CRITICAL}
  tags: [PHASE_1]

- id: FX_011
  domain: FX & Multi-Currency
  name: Base Currency Definition and Consistency
  type: DATA_QUALITY
  tier: 1
  description: |
    Fund base currency (reporting currency) must be consistent. All NAV, AUM, fees report
  inputs:
    - fund_config.base_currency
    - nav_log.currency
  logic_pseudocode: |
    for each nav_log:
      if nav_log.currency != fund_config.base_currency:
        flag NAV_CURRENCY_MISMATCH
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: FX_012
  domain: FX & Multi-Currency
  name: FX Spot Rate Historical Tracking
  type: DETERMINISTIC
```

```
  tier: 2
  description: |
    Maintain historical FX rates used for each trade. Enable audit trail and forensic ana
  inputs:
    - trade.trade_date
    - trade.fx_rate_used
    - trade.fx_rate_source
  logic_pseudocode: |
    for each trade with fx_conversion:
      historical_rate_record = fx_rate_history[trade_date, pair]
      if historical_rate_record not documented:
        flag FX_RATE_NOT_RECORDED_FOR_AUDIT
  severity: {level: MEDIUM}
  tags: [OPTIONAL, PHASE_1]
```

DOMAIN G: SUSPENSE & EXCEPTION MANAGEMENT (12 Rules)

```
- id: SUS_001
  domain: Suspense & Exceptions
  name: Unmatched Cash Detection
  type: DETERMINISTIC
  tier: 1
  description: |
    Flag cash entries that cannot be matched to any trade or known corporate action.
    Move to suspense register for investigation.
  inputs:
    - bank_txns[value_date]
    - broker_trades (all)
    - corp_actions (all, cash-related)
  logic_pseudocode: |
    for each bank_txn:
      matching_trade = find_matching_trade(bank_txn)
      matching_ca = find_matching_ca_payment(bank_txn)

      if matching_trade is None and matching_ca is None:
        move_to_suspense_register(bank_txn, reason="UNMATCHED_CASH")
  severity: {level: HIGH, impact: "Cash reconciliation break; requires investigation"}
  tags: [CORE, PHASE_1]

- id: SUS_002
  domain: Suspense & Exceptions
  name: Unmatched Security Detection
  type: DETERMINISTIC
  tier: 1
  description: |
    Flag securities in custodian holdings with no corresponding internal trade.
    May indicate missing trade record, dividend reinvestment, or data error.
  inputs:
    - custodian_holdings[isin].quantity: >0
    - internal_trades (all)
  logic_pseudocode: |
    for each custodian_holding:
      internal_holdings = find_internal_holding(isin)
      if internal_holdings.quantity == 0:
```

```
        move_to_suspense_register(
          security=isin,
          reason="UNMATCHED_SECURITY",
          custodian_qty=custodian_holdings.quantity
        )
  severity: {level: MEDIUM}
  tags: [CORE, PHASE_1]

- id: SUS_003
  domain: Suspense & Exceptions
  name: Amount Variance Beyond Tolerance
  type: TOLERANCE
  tier: 2
  description: |
    Trade ↔ Cash match where amount differs, but variance is beyond system tolerance.
    Escalate to manual review (could be fee adjustment, partial fill, or genuine error).
  inputs:
    - trade.net_amount
    - bank_txn.amount
    - tolerance_abs, tolerance_pct
  logic_pseudocode: |
    diff = abs(trade.net_amount - bank_txn.amount)
    if diff > tolerance_abs and (diff / trade.net_amount) > tolerance_pct:
      if not already_flagged:
        create_exception(
          type="AMOUNT_VARIANCE_BEYOND_TOLERANCE",
          trade_id, txn_id, variance_amount=diff
        )
  suggested_tolerances:
    amount_abs: 2
    amount_pct: 0.5
  severity: {level: MEDIUM}
  tags: [CORE, PHASE_1]

- id: SUS_004
  domain: Suspense & Exceptions
  name: Late Settlement Detection
  type: EDGE_CASE
  tier: 2
  description: |
    If a trade settlement is delayed >T+3, flag for custodian follow-up.
  inputs:
    - trade.settlement_date
    - trade.status: UNSETTLED (if current_date > settlement_date + 3)
  logic_pseudocode: |
    for each trade where status == UNSETTLED:
      days_since_settlement_date = current_date - trade.settlement_date
      if days_since_settlement_date > 3:
        create_exception(
          type="LATE_SETTLEMENT",
          trade_id,
          days_delayed=days_since_settlement_date
        )
  suggested_tolerances:
    date_offset_days: 3
  severity: {level: HIGH}
```

```
  tags: [CORE, PHASE_1]

- id: SUS_005
  domain: Suspense & Exceptions
  name: Partial Fill Reconciliation
  type: EDGE_CASE
  tier: 2
  description: |
    If a buy/sell order is partially filled, ensure each leg is tracked.
    Flag if one leg is missing.
  inputs:
    - broker_trades[parent_order_id]
    - total_qty_ordered
    - total_qty_filled
  logic_pseudocode: |
    for each parent_order:
      filled_legs = [t for t in trades[parent_order] if t.status == SETTLED]
      unfilled_qty = total_ordered - sum(filled_legs.quantity)

      if unfilled_qty > 0:
        create_exception(
          type="PARTIAL_FILL_PENDING",
          parent_order_id,
          unfilled_qty
        )
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: SUS_006
  domain: Suspense & Exceptions
  name: Dividend / Interest Receivable Aging
  type: TOLERANCE
  tier: 2
  description: |
    Track accrued dividend/interest receivables. If not received within grace period
    after payment date, flag for follow-up.
  inputs:
    - corp_actions[type in [DIVIDEND, INTEREST]].payment_date
    - bank_txns[description contains dividend/interest]
    - current_date
  logic_pseudocode: |
    for each dividend_ca:
      if current_date > payment_date + grace_period:
        cash_received = find_cash_entry(dividend_ca)
        if cash_received is None:
          create_exception(
            type="DIVIDEND_NOT_RECEIVED",
            isin,
            payment_date,
            days_overdue=current_date - payment_date
          )
  suggested_tolerances:
    date_offset_days: 5
  severity: {level: MEDIUM}
  tags: [PHASE_1]
```

```
- id: SUS_007
  domain: Suspense & Exceptions
  name: Reconciliation Break Aging Report
  type: DETERMINISTIC
  tier: 2
  description: |
    Group unresolved breaks by age (1-day, 3-day, 7-day, >7-day).
    Use for SLA tracking and ops management.
  inputs:
    - exceptions[all].creation_date
    - current_date
  logic_pseudocode: |
    for each exception:
      days_open = current_date - creation_date
      age_bucket = categorize_age(days_open)
      #  "< 1 day", "1-3 days", "3-7 days", "> 7 days"
      count_by_bucket
  severity: {level: LOW, impact: "Reporting / SLA tracking"}
  tags: [PHASE_1]

- id: SUS_008
  domain: Suspense & Exceptions
  name: Duplicate Trade Detection
  type: DATA_QUALITY
  tier: 1
  description: |
    Flag if same trade is recorded twice (same broker, amount, date).
    Prevents double-settlement or double-cash movement.
  inputs:
    - broker_trades[all]
  logic_pseudocode: |
    for each trade:
      duplicates = find_trades(
        broker=trade.broker,
        isin=trade.isin,
        quantity=trade.quantity,
        price=trade.price,
        trade_date=trade.trade_date,
        trade_time ~ trade.execution_time +/- 1 minute
      )
      if len(duplicates) > 1:
        create_exception(
          type="DUPLICATE_TRADE",
          trade_ids=[t.id for t in duplicates]
        )
  severity: {level: CRITICAL}
  tags: [CORE, PHASE_1]

- id: SUS_009
  domain: Suspense & Exceptions
  name: Reversal Not Matched to Original Trade
  type: DATA_QUALITY
  tier: 1
  description: |
    If a trade reversal exists (cancelled trade), ensure it references the original trade
  inputs:
```

```
    - broker_trades[status == CANCELLED]
    - broker_trades.original_trade_id or reversal_reference
  logic_pseudocode: |
    for each cancelled_trade:
      if cancelled_trade.original_trade_id is None:
        create_exception(
          type="REVERSAL_NOT_LINKED",
          cancelled_trade_id,
          reason="Cannot trace to original trade"
        )
  severity: {level: HIGH}
  tags: [CORE, PHASE_1]

- id: SUS_010
  domain: Suspense & Exceptions
  name: Zero-Quantity or Zero-Amount Transactions
  type: DATA_QUALITY
  tier: 1
  description: |
    Flag trades or cash entries with zero quantity or zero amount (data entry error).
  inputs:
    - broker_trades.quantity
    - broker_trades.gross_amount
    - bank_txns.amount
  logic_pseudocode: |
    for each trade:
      if trade.quantity == 0 or trade.gross_amount == 0:
        create_exception(
          type="ZERO_VALUE_TRANSACTION",
          trade_id
        )

    for each bank_txn:
      if bank_txn.amount == 0:
        create_exception(
          type="ZERO_AMOUNT_CASH_ENTRY",
          txn_id
        )
  severity: {level: MEDIUM}
  tags: [PHASE_1]

- id: SUS_011
  domain: Suspense & Exceptions
  name: Manual Override Audit Trail
  type: DETERMINISTIC
  tier: 1
  description: |
    When ops manually resolves an exception (override), log it with user ID, reason, and
    Enable forensic audit.
  inputs:
    - exceptions[resolved_manually].resolution_notes
    - exceptions.resolved_by_user_id
    - exceptions.resolved_date
    - exceptions.approval_status
  logic_pseudocode: |
    for each manually_resolved_exception:
```

```
      if resolved_by_user_id is None:
        flag OVERRIDE_NOT_LOGGED_WITH_USER
      elif approval_status != APPROVED:
        flag OVERRIDE_NOT_APPROVED
  severity: {level: HIGH, impact: "Compliance, audit"}
  tags: [CORE, PHASE_1]

- id: SUS_012
  domain: Suspense & Exceptions
  name: Recon Break Summary Dashboard
  type: DETERMINISTIC
  tier: 2
  description: |
    Generate summary: total breaks, by domain, by severity, by age.
    Use for daily ops standup and reporting.
  inputs:
    - exceptions[all]
  logic_pseudocode: |
    # Aggregation/reporting; not a validation rule per se
    # Group by: domain, severity, age, status (open/resolved)
    # Count and trending
  severity: {level: LOW}
  tags: [PHASE_1, REPORTING]
```

SECTION 3 — CORPORATE ACTIONS & SPECIAL DAYS

3.1 Corporate Actions Coverage

CA Type	Quantity Impact	Price Impact	Cost Basis Impact	Cash Impact
Split (5:1)	qty × 5	price ÷ 5	cost_per_share ÷ 5	None
Reverse Split (1:10)	qty ÷ 10	price × 10	cost_per_share × 10	None
Bonus (1:5)	qty × 1.2 (20% addition)	price ÷ 1.2	cost_base unchanged; new shares cost = 0	None
Cash Dividend (₹10/share)	None	None	None	cash_in = qty × ₹10
Rights Issue (1:4)	qty + rights_allotted	new shares at rights_price	cost_base split	Potential cash for subscription
Merger (ABC → XYZ)	qty_abc → 0; qty_xyz += qty_abc × exch_ratio	new ISIN, new price	new cost basis	None (unless cash component)

3.2 Rules for Each CA Type

Handled in Section 2, Domain E (Rules CA_001–CA_012).

3.3 Ex-Date, Record-Date, Payment-Date Logic

```
Timeline: Trade Date → Ex-Date → Record Date → Payment Date

On Ex-Date:
  ├─ Quantity adjustment (if split/bonus)
  ├─ Holdings snapshot taken for dividend
  └─ Reconciliation rule: holdings[ex_date] should reflect adjustment

Between Ex-Date and Record-Date:
  └─ Accrued dividend / interest (if not yet paid)

On Record-Date:
  └─ Eligibility confirmed; payment list finalized

On Payment-Date:
  ├─ Cash posted to bank account
  └─ Reconciliation rule: bank_txn.amount = holdings[ex_date] × dividend_per_share

Post-Payment-Date (grace period +5 days):
  └─ If not received, flag for follow-up
```

SECTION 4 — FX & MULTI-CURRENCY LOGIC

4.1 FX Reconciliation Scenarios

Scenario 1: USD Trade, INR Settlement (US Fund Buying India Stock)

```
Trade:  BUY 100 Apple @ USD 150 on 1-Jan
        Trade currency: USD
        Settlement currency: USD

        Internal holding: 100 Apple (USD)

Fund base currency: INR

At T+2 (settlement):
  Broker: Pay USD 15,000 (net)
  Bank: Debit ₹1,252,500 (assuming USD 1 = ₹ 83.50 spot)

Reconciliation:
  - Verify: ₹1,252,500 ÷ 83.50 = USD 15,000 ✓
  - Verify FX rate source (custodian vs market)
  - Track FX variance if deal rate ≠ spot rate

NAV Impact:
```

```
  - Holdings: 100 shares Apple @ USD 150 = USD 15,000
  - Base currency NAV: USD 15,000 × 83.50 = ₹1,252,500
  - Each day, re-mark at current spot rate for unrealized FX P&L
```

Scenario 2: Hedged FX Forward

```
Trade:  BUY 100 Apple @ USD 150
        Hedge: FX Forward contract locked at USD 1 = ₹83.45

Reconciliation:
  - Match trade.net_amount_usd (15,000) to forward.notional_amount ✓
  - Match forward.settlement_date to trade.settlement_date ✓
  - Calculate FX P&L: (83.45 - 83.50) × 15,000 = -₹75 loss
  - Verify forward is marked-to-market daily until settlement
  - At settlement, verify cash debit matches forward rate (₹1,252,250)
```

Scenario 3: Multi-Currency Fund (Mixed Holdings)

```
Holdings:
  ├─ 100 Apple @ USD 150 = USD 15,000
  ├─ 50 ASML (EUR) @ EUR 600 = EUR 30,000
  └─ ₹10,00,000 cash

FX Rates (EOD):
  ├─ USD 1 = ₹ 83.50
  └─ EUR 1 = ₹ 91.00

AUM Calc (base currency INR):
  = (USD 15,000 × 83.50) + (EUR 30,000 × 91.00) + ₹10,00,000
  = ₹12,52,500 + ₹27,30,000 + ₹10,00,000
  = ₹49,82,500

Reconciliation:
  - Verify each currency converted at agreed FX rate (custodian rate)
  - Validate no missing conversions
  - Track daily unrealized FX P&L
```

4.2 FX Rules (Covered in Section 2, Domain F)

- **FX_001–FX_012:** See Domain F above

SECTION 5 — EXCEPTION MANAGEMENT & WORKFLOWS

5.1 Exception Lifecycle

```
Step 1: DETECTION
  ├─ Rule engine identifies a break
  ├─ Exception created with: type, severity, entities involved, rule_id
  └─ Timestamp: detection_datetime

Step 2: AUTO-RESOLUTION ATTEMPT (if applicable)
  ├─ Some breaks auto-resolved by deterministic rules (e.g., duplicate override)
  ├─ Marked as: STATUS = RESOLVED (auto)
  ├─ Logged in learning_events for future self-healing
  └─ No ops escalation needed

Step 3: ESCALATION TO OPS (if not auto-resolved)
  ├─ Break pushed to ops queue (Slack, email, dashboard)
  ├─ Severity determines urgency:
  │  ├─ CRITICAL: immediate escalation, daily SLA
  │  ├─ HIGH: escalation within 3 hours, 1-day SLA
  │  ├─ MEDIUM: next business day, 3-day SLA
  │  └─ LOW: batch review, 7-day SLA
  ├─ Assigned to ops analyst or fund PM (depending on severity + domain)
  └─ STATUS = ESCALATED_TO_OPS

Step 4: MANUAL INVESTIGATION & RESOLUTION
  ├─ Ops analyst:
  │  ├─ Reviews break context (trade details, custodian reports, etc.)
  │  ├─ Investigates root cause (missing trade? Timing issue? Data error?)
  │  ├─ Proposes resolution (confirm trade, create fee adjustment, etc.)
  │  └─ Documents findings in exception notes
  ├─ If simple (e.g., timing): ops closes with explanation
  ├─ If complex (e.g., custodian error): escalates to PM or compliance
  └─ STATUS = IN_INVESTIGATION

Step 5: RESOLUTION / APPROVAL
  ├─ Ops applies fix (create offsetting entry, journal entry, etc.)
  ├─ For material breaks ($ or regulatory impact): escalate to PM/Compliance for approva
  ├─ Approval gates:
  │  ├─ Timing breaks: ops closure
  │  ├─ Fee discrepancies: PM review (if >threshold)
  │  ├─ Regulatory implications: Compliance review
  │  └─ PnL-impacting breaks: PM + Compliance approval
  └─ STATUS = RESOLVED or APPROVED_AND_RESOLVED

Step 6: CLOSURE & LEARNING
  ├─ Exception marked CLOSED
  ├─ Resolution logged: resolution_type, resolution_timestamp, resolved_by_user_id, appr
  ├─ Break pattern analyzed by AI/LLM (Tier-3):
  │  ├─ Was this a known vendor quirk? Update tolerance or add broker-specific rule.
  │  ├─ Is there a pattern? Suggest new rule to reduce future breaks.
  │  └─ Was manual fix applied? Log in learning_events for self-healing.
  └─ Exception retained in audit trail (never deleted)

Aging Tracking:
  ├─ If open > SLA: flag for escalation to PM
  └─ Daily report: breakdown of open exceptions by domain, age, severity
```

5.2 Exception Categories & Severity

Category	Definition	Typical Severity	Examples
DATA	Data entry error, missing fields, invalid format	MEDIUM	Zero qty, negative amount, invalid ISIN
TIMING	Settlement delay, date mismatch	MEDIUM–HIGH	Cash arrived on T+4 instead of T+2
ECONOMIC	Amount mismatch, fee variance	HIGH	Trade ↔ Cash off by ₹5,000
CORPORATE_ACTION	CA not applied, dividend mismatch	CRITICAL	Split not reflected in qty
COUNTERPARTY	Broker/custodian issue	MEDIUM–HIGH	Broker settlement not matching report
SYSTEM	Software bug, ingestion failure	CRITICAL	Duplicate trade, corrupted field
REGULATORY	Compliance or regulatory reporting issue	CRITICAL	NAV calculation error, fee overcharge

5.3 Auto-Close vs Manual Queue

Scenario	Action	SLA
Exact amount match, on-time timing	Auto-close (no break)	N/A
Amount within tolerance, timing within ±2 days	Auto-close (no break)	N/A
Late settlement (T+4), but amount matches	Flag TIMING, auto-close with note	3 days
Amount variance >tolerance but <1% AND on-time timing	Escalate to ops for fee review	1 day
Unmatched cash or security	Escalate to ops queue	1 day (CRITICAL), 3 days (HIGH)
Cancelled trade but cash not reversed	Auto-flag, escalate to ops	1 day
CA not applied	Escalate to PM + ops	SAME DAY
NAV variance >threshold	Escalate to PM + Compliance	SAME DAY

SECTION 6 — PARTITIONING LOGIC: RULE ENGINE vs AI vs SELF-HEALING

6.1 Tier Classification Summary

Tier	Live In	Trigger	Example Rules	Coverage
Tier-1: Deterministic	RULE_ENGINE (SQL/Python)	Every run	Exact match, accounting identities, status alignment	~70% of breaks
Tier-2: Tolerance	RULE_ENGINE (SQL/Python)	Every run	Amount variance, date offsets, FX rounding	~15% of breaks
Tier-3: Data Quality	RULE_ENGINE (SQL/Python)	Every run	Field validation, impossible states, format checks	~10% of breaks
Tier-4: AI/Fuzzy	AI_LAYER (LLM)	On residual breaks	Narrative matching, broker quirks, partial fills	~5% of breaks
Self-Healing	RULE_MEMORY + AI	Batch (nightly)	Pattern detection, tolerance tuning, new rules	Continuous improvement

6.2 Proposed Python Rule Engine API (25+ Core Functions)

```
# File: backend/rule_engine/reconciliation_engine.py

class ReconciliationRuleEngine:
    """Core rule engine for multi-domain reconciliation."""

    # ============ MATCHING FUNCTIONS ============

    def match_trade_to_cash(
        self,
        trade: Trade,
        candidate_cash: List[CashTxn],
        tolerance_abs: float = 1.0,
        tolerance_pct: float = 0.1,
        date_offset_days: int = 2
    ) -> Optional[MatchResult]:
        """
        Match a settled trade to a cash movement.
        Returns: MatchResult(matched_cash_id, confidence_score, reason)
        """
        pass

    def match_holdings_to_custodian(
        self,
        internal_holding: Position,
        custodian_holding: Position,
        qty_tolerance: int = 0
    ) -> MatchResult:
        """
        Reconcile internal vs custodian holdings by ISIN.
        Returns: MatchResult with qty_variance, price_variance, etc.
        """
```

```
        pass

    def match_nav_internal_to_custodian(
        self,
        internal_nav: NAVLog,
        custodian_nav: NAVLog,
        tolerance_pct: float = 1.0
    ) -> MatchResult:
        """
        Validate internal NAV vs custodian NAV within tolerance.
        """
        pass

    # ============ CASH RECONCILIATION ============

    def reconcile_trade_cash_exact(
        self,
        trade: Trade,
        bank_txns: List[CashTxn],
        settlement_window: int = 2
    ) -> List[Reconciliation]:
        """
        RULE TRADE_CASH_001: Exact amount match (DETERMINISTIC).
        Returns list of reconciliations (matched or flagged breaks).
        """
        pass

    def reconcile_partial_fill(
        self,
        parent_order_id: str,
        trades: List[Trade],
        bank_txns: List[CashTxn],
        tolerance_abs: float = 1.0
    ) -> List[Reconciliation]:
        """
        RULE TRADE_CASH_002: Handle partial fills; match each leg.
        """
        pass

    def reconcile_trade_reversal(
        self,
        original_trade: Trade,
        reversal_trade: Trade,
        bank_txns: List[CashTxn]
    ) -> Reconciliation:
        """
        RULE TRADE_CASH_005 / TRADE_CASH_007: Verify reversal is reflected in cash.
        """
        pass

    def validate_buy_sell_direction(
        self,
        trade: Trade,
        bank_txn: CashTxn
    ) -> Reconciliation:
        """
```

```
        RULE TRADE_CASH_008: Validate BUY → debit, SELL → credit.
        """
        pass

    def validate_fee_decomposition(
        self,
        trade: Trade,
        broker_fee_schedule: Dict
    ) -> Reconciliation:
        """
        RULE TRADE_CASH_003 / TRADE_CASH_009 / TRADE_CASH_013:
        Validate brokerage, STT, GST calculations.
        """
        pass

    # ============ HOLDINGS RECONCILIATION ============

    def reconcile_position_quantity(
        self,
        isin: str,
        as_of_date: date,
        internal_qty: int,
        custodian_qty: int,
        qty_tolerance: int = 0
    ) -> Reconciliation:
        """
        RULE POS_CUST_001: Holdings qty match.
        """
        pass

    def apply_corporate_action_adjustment(
        self,
        position: Position,
        corp_action: CorporateAction
    ) -> Position:
        """
        RULE POS_CUST_002 / CA_001–CA_012:
        Apply split, bonus, dividend impact to position.
        """
        pass

    def validate_pledged_free_split(
        self,
        position: Position
    ) -> Reconciliation:
        """
        RULE POS_CUST_003: Verify pledged_qty + free_qty = total_qty.
        """
        pass

    def reconcile_valuation(
        self,
        internal_value: float,
        custodian_value: float,
        tolerance_pct: float = 2.0
    ) -> Reconciliation:
```

```
        """
        RULE POS_CUST_004: Validate valuation variance < threshold.
        """
        pass

    def detect_new_security(
        self,
        position: Position,
        days_since_trade: int,
        settlement_window: int = 2
    ) -> Reconciliation:
        """
        RULE POS_CUST_005: Flag if internal has qty but custodian doesn't.
        """
        pass

    def detect_unexpected_security(
        self,
        custodian_holding: Position,
        internal_holdings: List[Position]
    ) -> Reconciliation:
        """
        RULE POS_CUST_006: Flag if custodian has security not in internal.
        """
        pass

    # ============ NAV & VALUATION ============

    def reconcile_nav_calculation(
        self,
        holdings: List[Position],
        cash_balance: float,
        reported_aum: float,
        tolerance_pct: float = 0.5
    ) -> Reconciliation:
        """
        RULE NAV_VAL_001: AUM = sum(holdings) + cash.
        """
        pass

    def validate_nav_per_unit(
        self,
        aum: float,
        total_units: int,
        reported_nav: float
    ) -> Reconciliation:
        """
        RULE NAV_VAL_002: NAV per unit = AUM / units.
        """
        pass

    def reconcile_internal_vs_custodian_nav(
        self,
        internal_nav: float,
        custodian_nav: float,
        tolerance_pct: float = 1.0
```

```
    ) -> Reconciliation:
        """
        RULE NAV_VAL_003: Validate NAV variance < tolerance.
        """
        pass

    def validate_mgt_fee_accrual(
        self,
        aum: float,
        fee_rate: float,
        days_elapsed: int,
        accrued_fee_on_books: float
    ) -> Reconciliation:
        """
        RULE NAV_VAL_007: Fee accrual = AUM × rate × (days / 365).
        """
        pass

    # ============ FX & MULTI-CURRENCY ============

    def reconcile_fx_settlement_amount(
        self,
        trade_amount_trade_ccy: float,
        fx_rate: float,
        settlement_amount_settlement_ccy: float,
        tolerance_pct: float = 0.1
    ) -> Reconciliation:
        """
        RULE FX_001: Validate settlement = trade_amount × fx_rate.
        """
        pass

    def match_fx_forward_to_trade(
        self,
        trade: Trade,
        fx_forward: FXForward
    ) -> Reconciliation:
        """
        RULE FX_003: Ensure forward contract exists and is matched.
        """
        pass

    def reconcile_multi_currency_aum(
        self,
        holdings_by_ccy: Dict[str, float],
        fx_rates: Dict[str, float],
        reported_aum_base_ccy: float,
        tolerance_pct: float = 0.5
    ) -> Reconciliation:
        """
        RULE FX_006: AUM (base) = sum(AUM_ccy × fx_rate_ccy_to_base).
        """
        pass

    # ============ CORPORATE ACTIONS ============
```

```
    def apply_split(
        self,
        position: Position,
        split_ratio_num: int,
        split_ratio_denom: int
    ) -> Position:
        """
        RULE CA_001 / CA_012: Apply split adjustment.
        """
        pass

    def apply_bonus(
        self,
        position: Position,
        bonus_ratio: float
    ) -> Position:
        """
        RULE CA_002: Apply bonus issue (qty increase, cost_base unchanged).
        """
        pass

    def reconcile_dividend_cash(
        self,
        holding_qty: int,
        dividend_per_share: float,
        expected_cash: float,
        actual_cash: float,
        tolerance_abs: float = 1.0
    ) -> Reconciliation:
        """
        RULE CA_004: Cash received = qty × dividend_per_share.
        """
        pass

    # ============ EXCEPTION MANAGEMENT ============

    def detect_unmatched_cash(
        self,
        bank_txn: CashTxn,
        trades: List[Trade],
        corp_actions: List[CorporateAction]
    ) -> Reconciliation:
        """
        RULE SUS_001: Flag cash with no matching trade or CA.
        """
        pass

    def detect_unmatched_security(
        self,
        custodian_holding: Position,
        internal_holdings: List[Position]
    ) -> Reconciliation:
        """
        RULE SUS_002: Flag security in custodian not in internal.
        """
        pass
```

```
    def detect_duplicate_trade(
        self,
        trade: Trade,
        all_trades: List[Trade],
        match_window_minutes: int = 1
    ) -> Reconciliation:
        """
        RULE SUS_008: Detect duplicate trades (same broker, qty, price, time~).
        """
        pass

    def detect_zero_value_transaction(
        self,
        trade: Trade
    ) -> Reconciliation:
        """
        RULE SUS_010: Flag trades with qty=0 or amount=0.
        """
        pass

    # ============ BATCH RECONCILIATION ============

    def run_daily_reconciliation(
        self,
        tenant_id: str,
        as_of_date: date,
        config: ReconciliationConfig
    ) -> ReconciliationReport:
        """
        Main entry point: run full reconciliation for tenant on date.
        Returns: matched reconciliations + flagged breaks.
        """
        pass

    def run_incremental_reconciliation(
        self,
        tenant_id: str,
        new_file_entities: List[Entity],
        as_of_date: date
    ) -> ReconciliationReport:
        """
        Incremental run when new file is uploaded (demo/testing).
        """
        pass

    # ============ CONFIGURATION & TOLERANCES ============

    def load_rule_config(
        self,
        tenant_id: str,
        as_of_date: date
    ) -> Dict[str, Any]:
        """
        Load tenant-specific rule tolerances, broker quirks, etc. from rule_definitions t
        """
```

```
        pass

    def update_rule_tolerance(
        self,
        rule_id: str,
        new_tolerance_abs: Optional[float] = None,
        new_tolerance_pct: Optional[float] = None,
        effective_date: date = today()
    ) -> None:
        """
        Update tolerance for a rule (e.g., based on self-healing analysis).
        """
        pass
```

6.3 Self-Healing Architecture

Phase 1: Log and Learn

```
class LearningEngine:
    """Capture manual fixes and identify patterns."""

    def log_manual_fix(
        self,
        exception_id: str,
        resolution_type: str,  # CONFIRMED, CREATED_ENTRY, WAIVED, etc.
        fields_adjusted: Dict[str, Tuple[old_value, new_value]],
        reason: str,
        resolved_by_user_id: str
    ) -> None:
        """
        Log manual resolution to learning_events table.
        Schema:
          learning_events(
            id, tenant_id, exception_id, rule_id, domain,
            resolution_type, fields_adjusted, reason,
            resolved_by_user_id, resolution_date, is_pattern
          )
        """
        pass

    def log_ai_override(
        self,
        exception_id: str,
        llm_proposal: Dict,
        human_choice: str,
        confidence_delta: float
    ) -> None:
        """
        Log cases where human disagreed with AI, for model retraining.
        """
        pass
```

Phase 2: Batch Analysis (Nightly)

```
class SelfHealingAgent:
    """Analyze patterns and propose rule updates."""

    def analyze_learning_events_batch(
        self,
        tenant_id: str,
        lookback_days: int = 30
    ) -> List[RuleImprovement]:
        """
        Nightly job:
        1. Query learning_events for past 30 days
        2. Group by rule_id, domain, broker, field_adjusted
        3. Identify patterns (e.g., "Broker X always off by INR 5 on STT")
        4. Return list of proposed improvements
        """
        # Pseudocode:
        # 1. SELECT COUNT(*), AVG(variance_amount), STD(variance)
        #    FROM learning_events
        #    WHERE domain = 'TRADE_CASH' AND rule_id = 'TRADE_CASH_003'
        #    AND resolution_type = 'FEE_ADJUSTMENT'
        #    GROUP BY broker, fee_type
        #
        # 2. If pattern emerges:
        #    - TRADE_CASH_003 consistently off for broker="Goldman" by avg INR 2–5
        #    → Propose: add broker-specific tolerance for Goldman STT calc
        #    → Confidence: HIGH if pattern occurs >5 times
        pass

    def generate_improved_rules(
        self,
        learning_summary: Dict
    ) -> List[RuleUpdate]:
        """
        For each pattern identified, call LLM to generate proposed rule update.
        Example prompt:

        "Looking at 30 days of manual fixes in the reconciliation engine:
         - Rule TRADE_CASH_003 (fee decomposition) has 12 manual fixes
         - Pattern: broker 'Goldman' consistently off by 2–5 rupees on GST calc
         - Root: Goldman uses a slightly different GST slab rounding
         - Current tolerance: ±0.5 rupees

         Propose a rule update:
         1. Add broker-specific tolerance for Goldman: ±5 rupees
         2. OR add logic to detect and auto-correct Goldman's rounding
         3. Which is safer? Why?"

        Returns JSON:
          {
            "rule_id": "TRADE_CASH_003",
            "broker": "Goldman",
            "current_tolerance_abs": 0.5,
            "proposed_tolerance_abs": 5,
            "confidence": 0.85,
```

```
            "rationale": "...",
            "requires_approval": true | false
          }
        """
        pass

    def apply_rule_update(
        self,
        rule_update: RuleUpdate,
        approved_by_user_id: str
    ) -> None:
        """
        Store approved rule update to rule_definitions table.
        Version control: keep history of all updates.
        """
        pass
```

Phase 3: Continuous Improvement Loop

```
Day 1:  Ops encounters break in TRADE_CASH_003 (Goldman GST calc)
        → Exception created, flagged to ops

Day 1:  Ops manually adjusts: GST amount = +5 rupees
        → Resolution logged to learning_events table
        → Similar pattern matches 11 other fixes in past 30 days

Day 2 (Nightly):
        SelfHealingAgent.analyze_learning_events_batch() runs
        → Detects: TRADE_CASH_003 + Goldman + GST = 12 fixes, pattern confirmed
        → Calls LLM to generate rule
```